

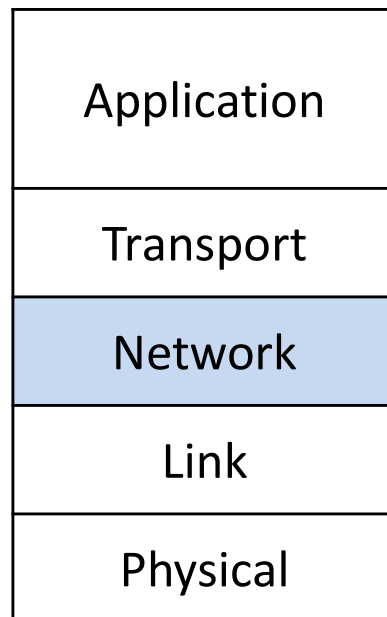
Routing

Dept. of Computer Science and Engineering
Sogang University

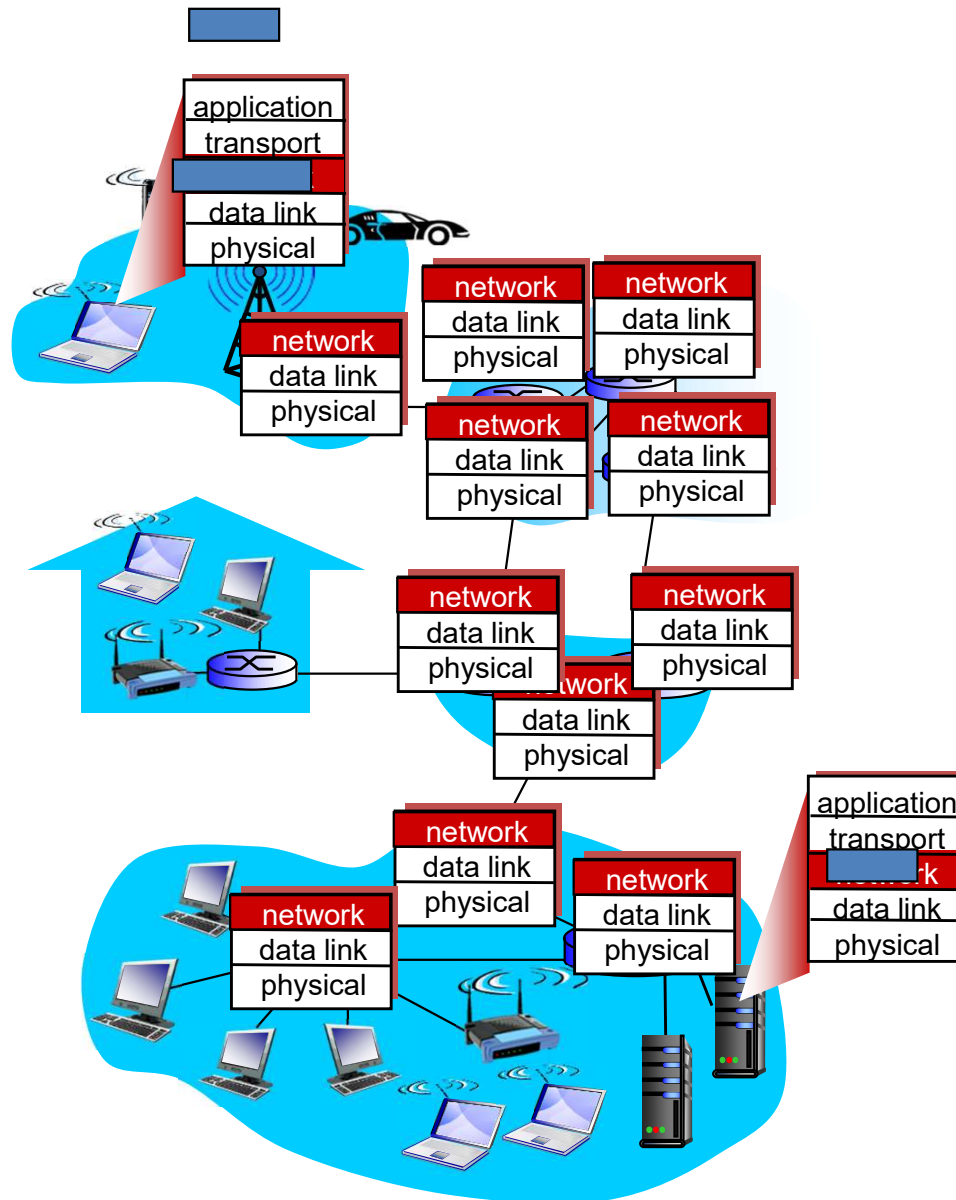
Overview

Network Layer

- Transports segments from sending to receiving host
- On sending side, encapsulates segments into datagrams
- On receiving side, delivers segments to transport layer
- Network layer protocols in every host, router
- Router examines header fields in all IP datagrams passing through



Network Layer

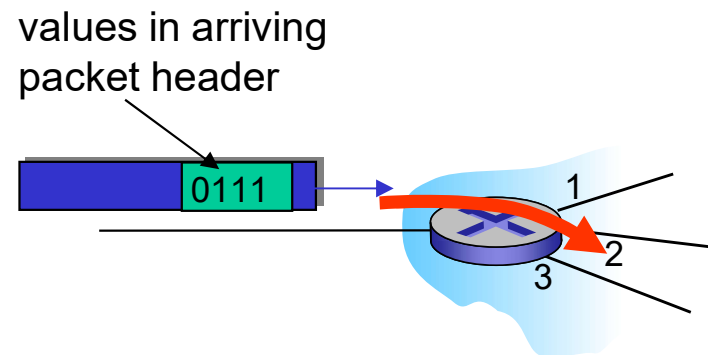


Network Layer Functions

- **Forwarding**: move packets from router's input to appropriate router output
 - analogy: process of getting through a single interchange
 - **DATA PLANE**
- **Routing**: determine route taken by packets from source to destination
 - analogy: process of planning trip from source to destination
 - **CONTROL PLANE**

Network Layer: Data Plane

- Local, per-router function
- Determines how datagram arriving on router input is forwarded to router output port
- Forwarding function

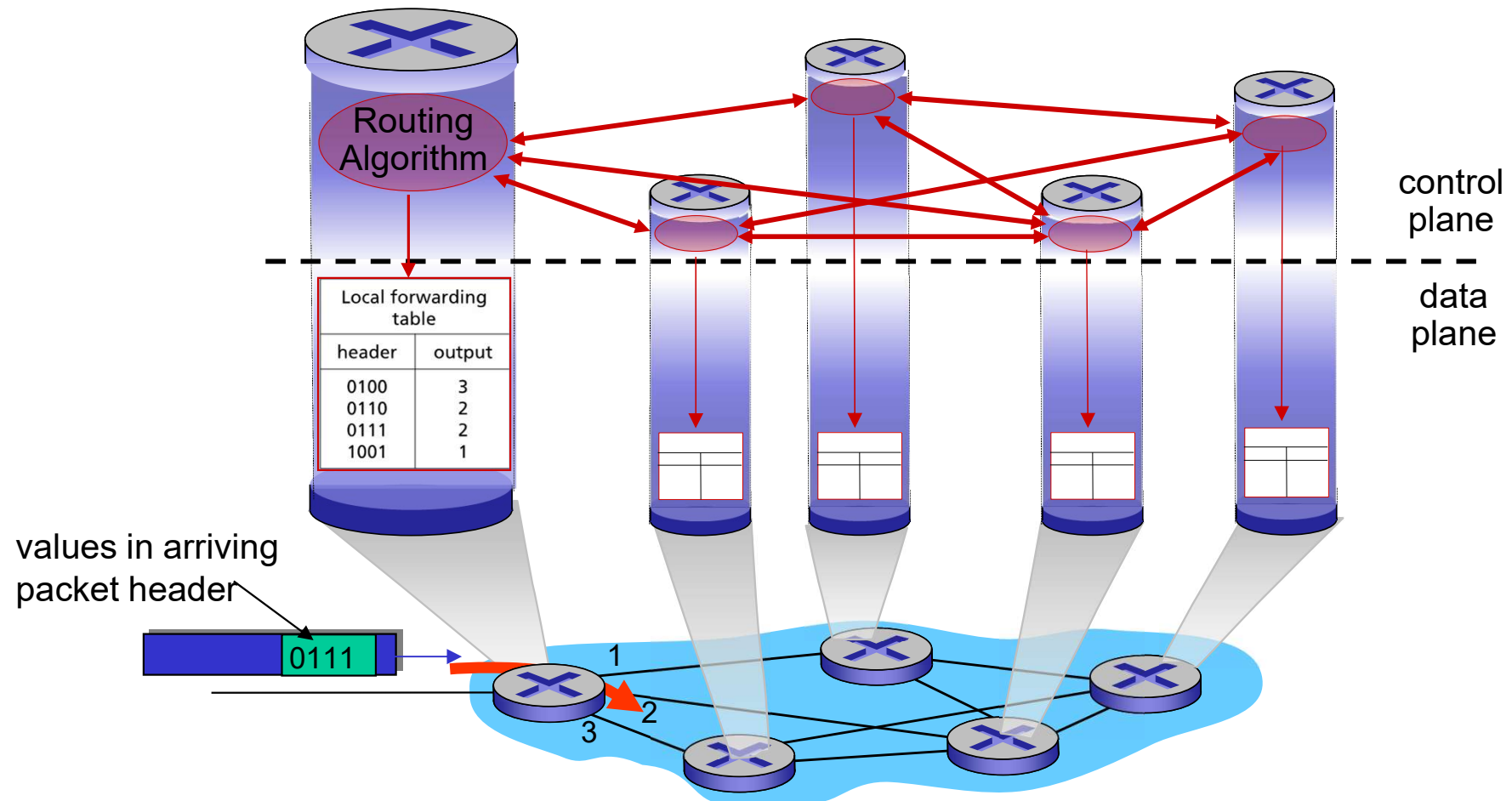


Network Layer: Control Plane

- Network-wide logic
- Determines how datagram is routed among routers along end-to-end path from source host to destination host
- Two control-plane approaches
 - traditional routing algorithms: implemented in routers
 - software-defined networking (SDN): implemented in (remote) servers

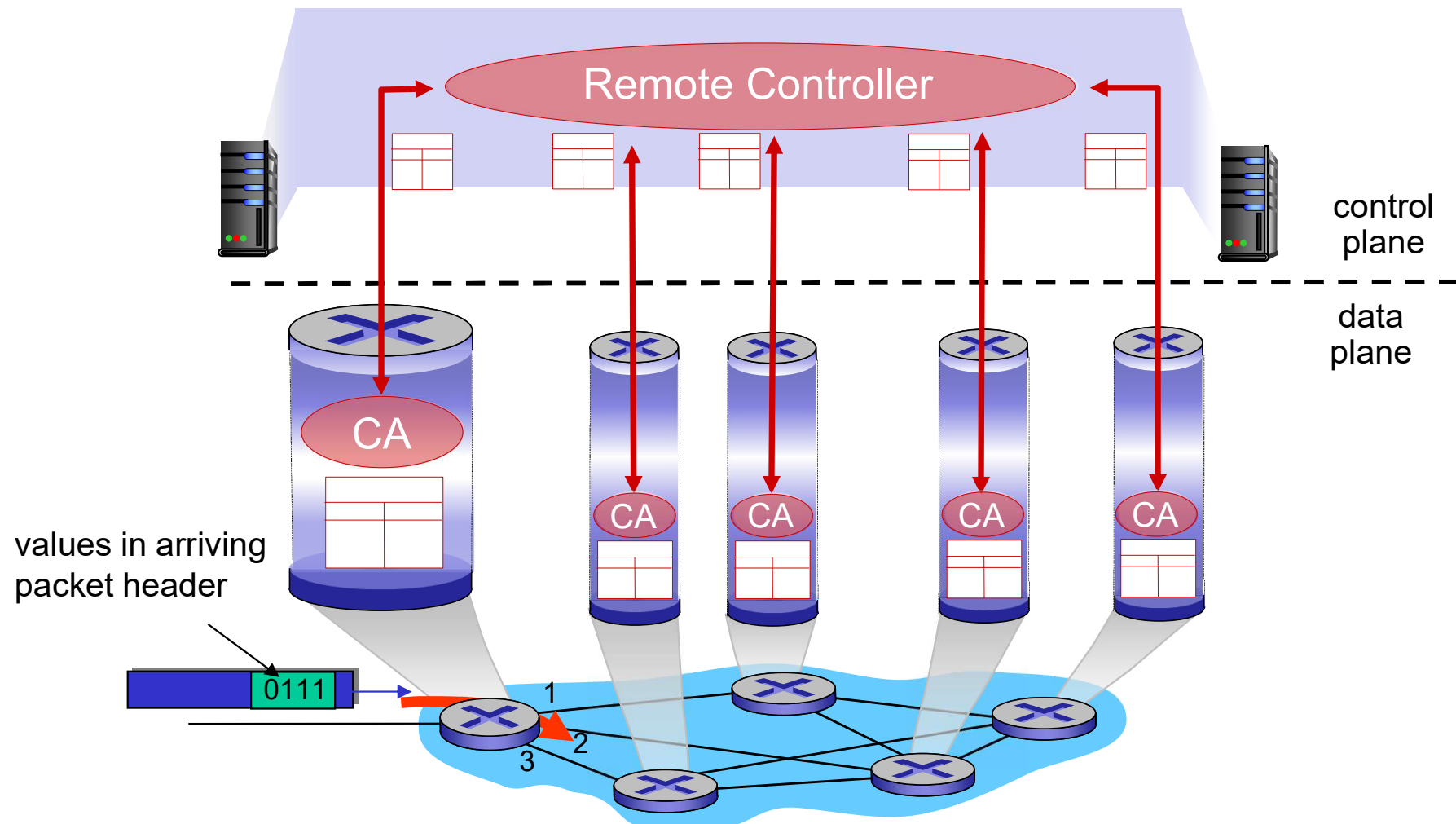
Per-Router Control Plane

- Individual routing algorithm components in each and every router interact in the control plane



Logically Centralized Control Plane

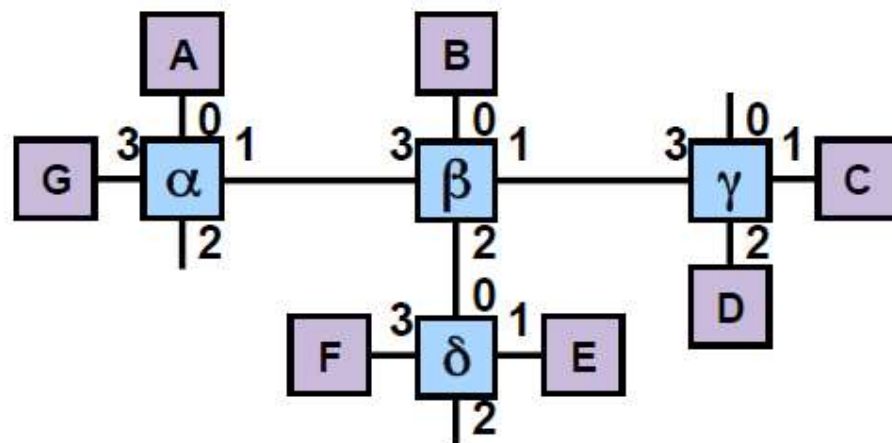
- A distinct (typically remote) controller interacts with local control agents (CAs)



Routing

Routing

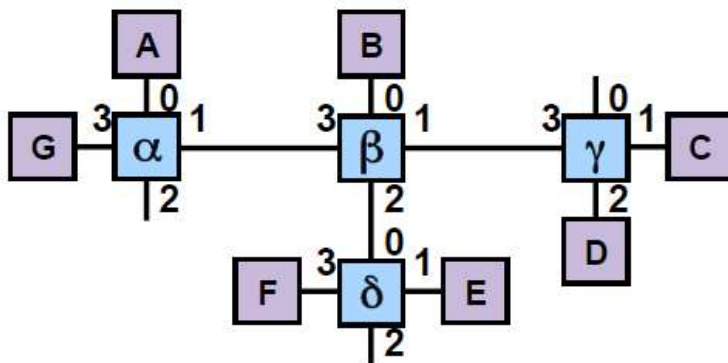
- Deciding path on which packets are forwarded
- Method for routers to construct the forwarding table
- In real routers, destinations are IP prefixes (e.g., 163.239.0.0/16); here we use single letters for simplicity



α 's Table	β 's Table	γ 's Table	δ 's Table
A 0	A 3	A 3	A 0
B 1	B 0	B 3	B 0
C 1	C 1	C 1	C 0
D 1	D 1	D 2	D 0
E 1	E 2	E 3	E 1
F 1	F 2	F 3	F 3
G 3	G 3	G 3	G 0

Routing

- Delivering a packet from A to F
 - Router α sends packet to output port 1
 - Router β sends packet to output port 2
 - Router δ sends packet to output port 3

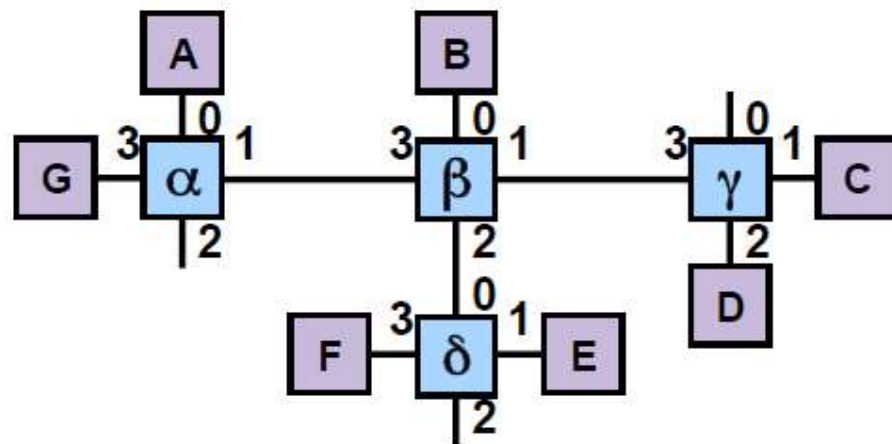


α 's Table β 's Table γ 's Table δ 's Table

A	0	A	3	A	3	A	0
B	1	B	0	B	3	B	0
C	1	C	1	C	1	C	0
D	1	D	1	D	2	D	0
E	1	E	2	E	3	E	1
F	1	F	2	F	3	F	3
G	3	G	3	G	3	G	0

Routing

- How does switch α know that the packet should go through output port 1 to reach F?



α 's Table β 's Table γ 's Table δ 's Table

A	0
B	1
C	1
D	1
E	1
F	1
G	3

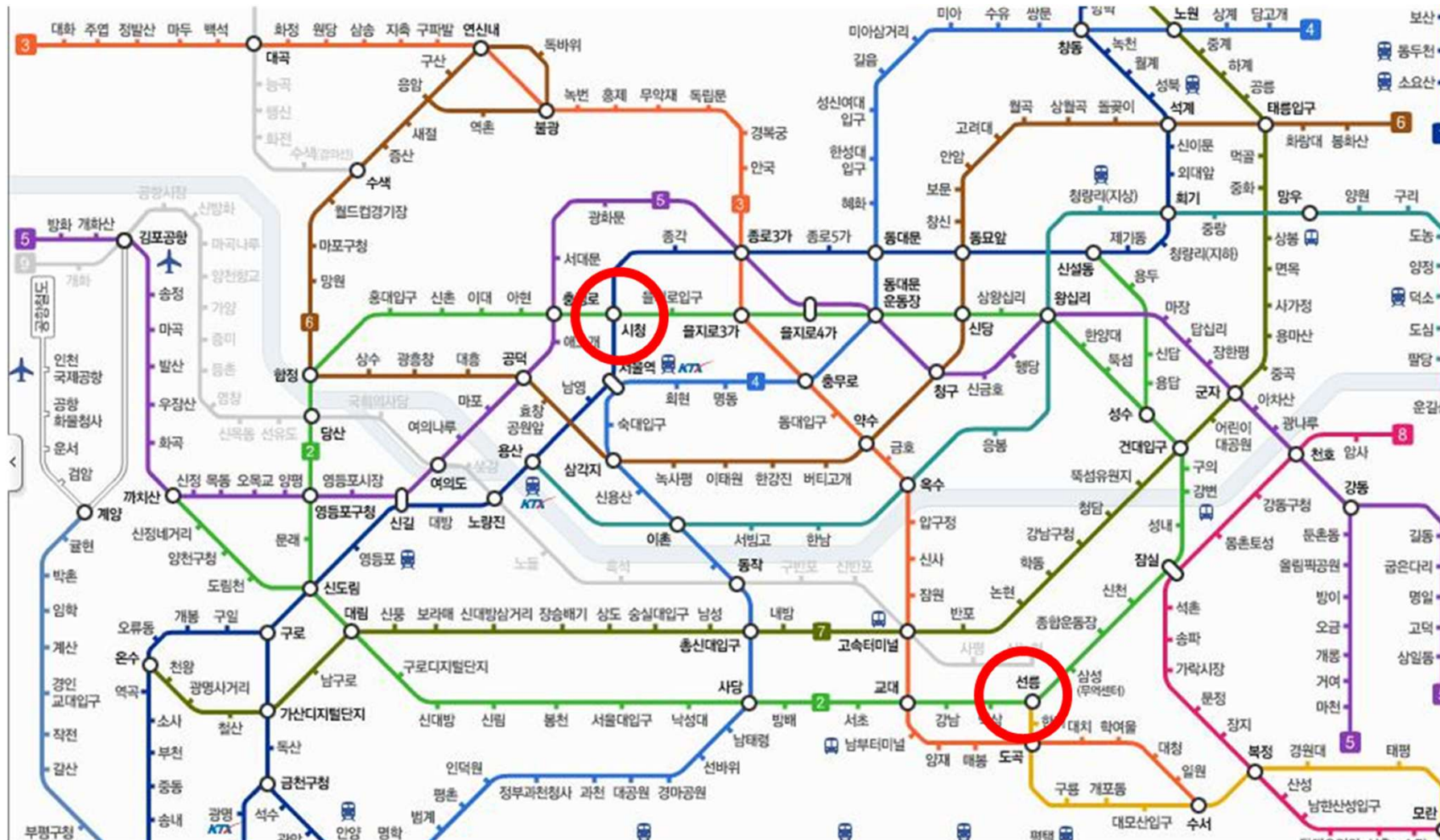
A	3
B	0
C	1
D	1
E	2
F	2
G	3

A	3
B	3
C	1
D	2
E	3
F	3
G	3

A	0
B	0
C	0
D	0
E	1
F	3
G	0

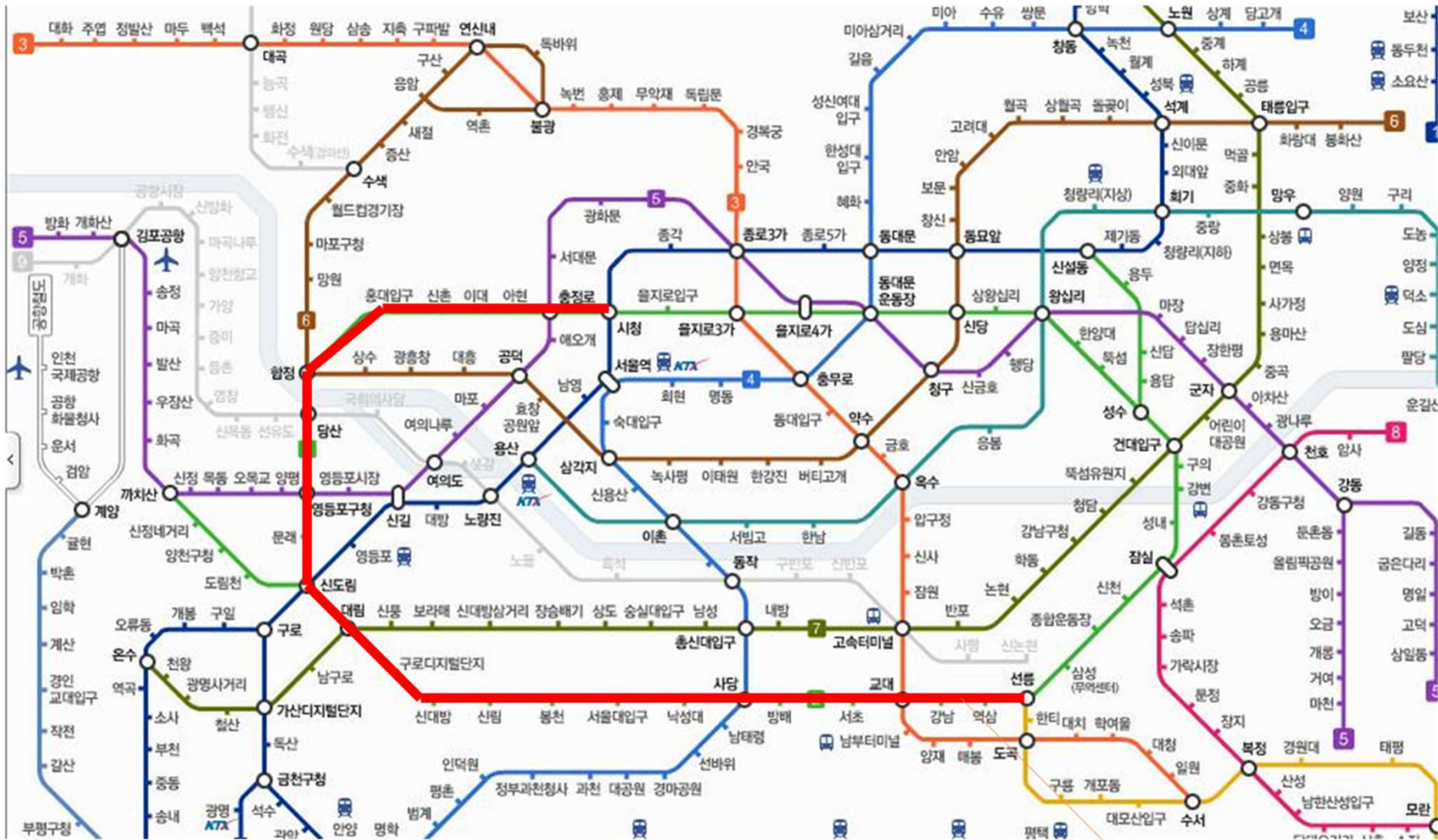
Routing

- I'm at 선릉 station. I want to go to 시청 station.



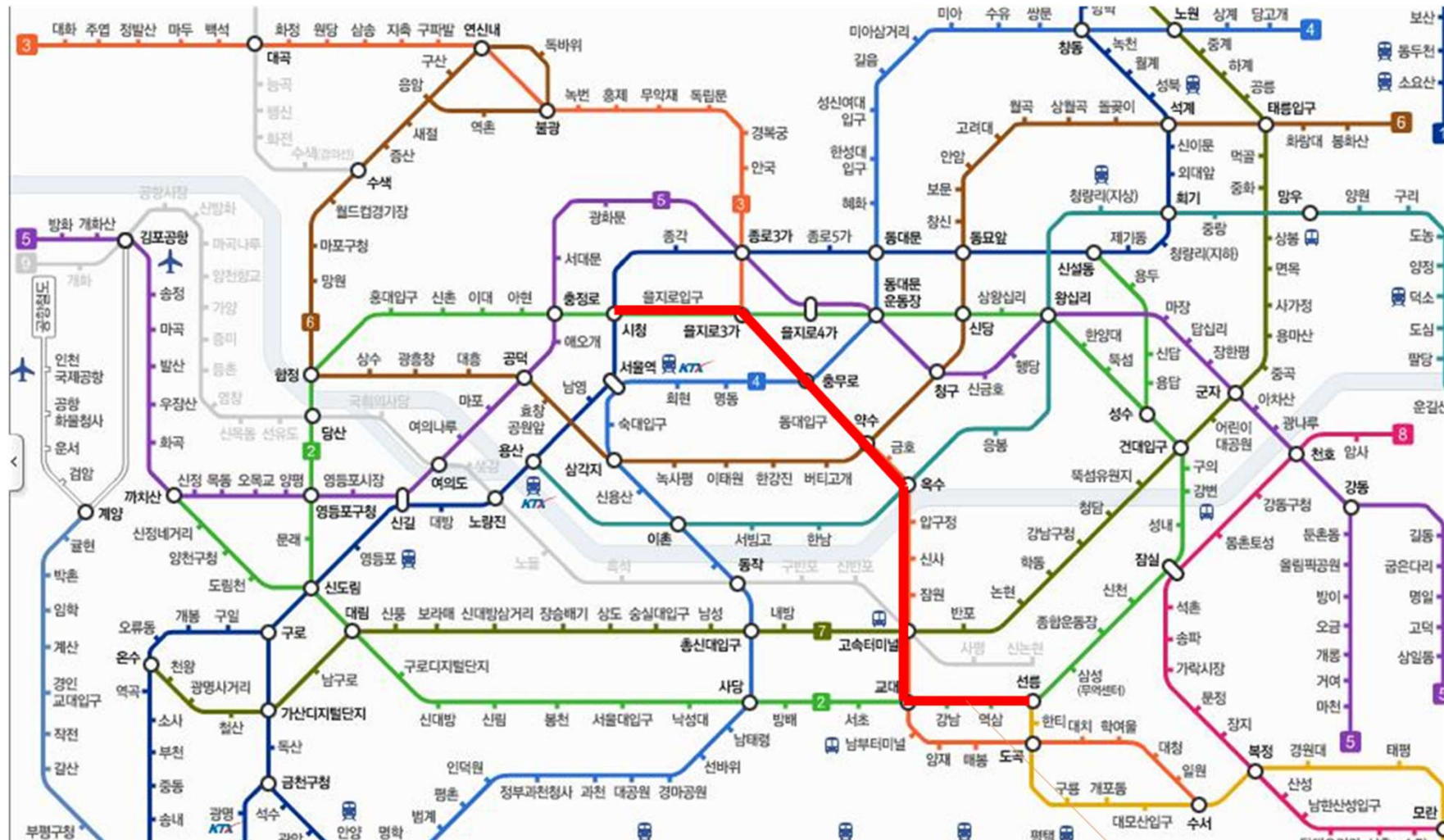
Routing

- 선릉 → 시청



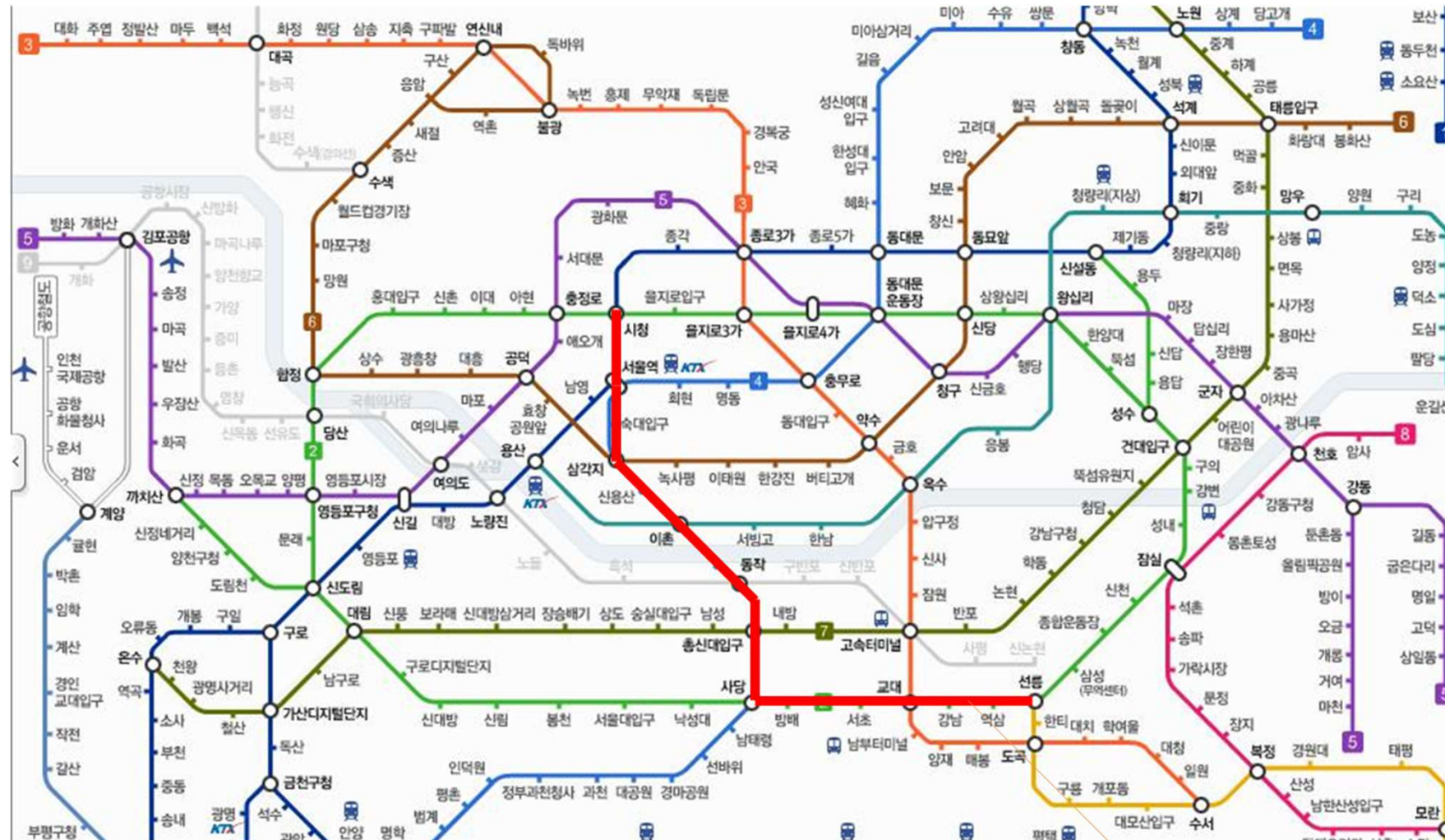
Routing

- 선릉 → 교대 → 을지로3가 → 시청



Routing

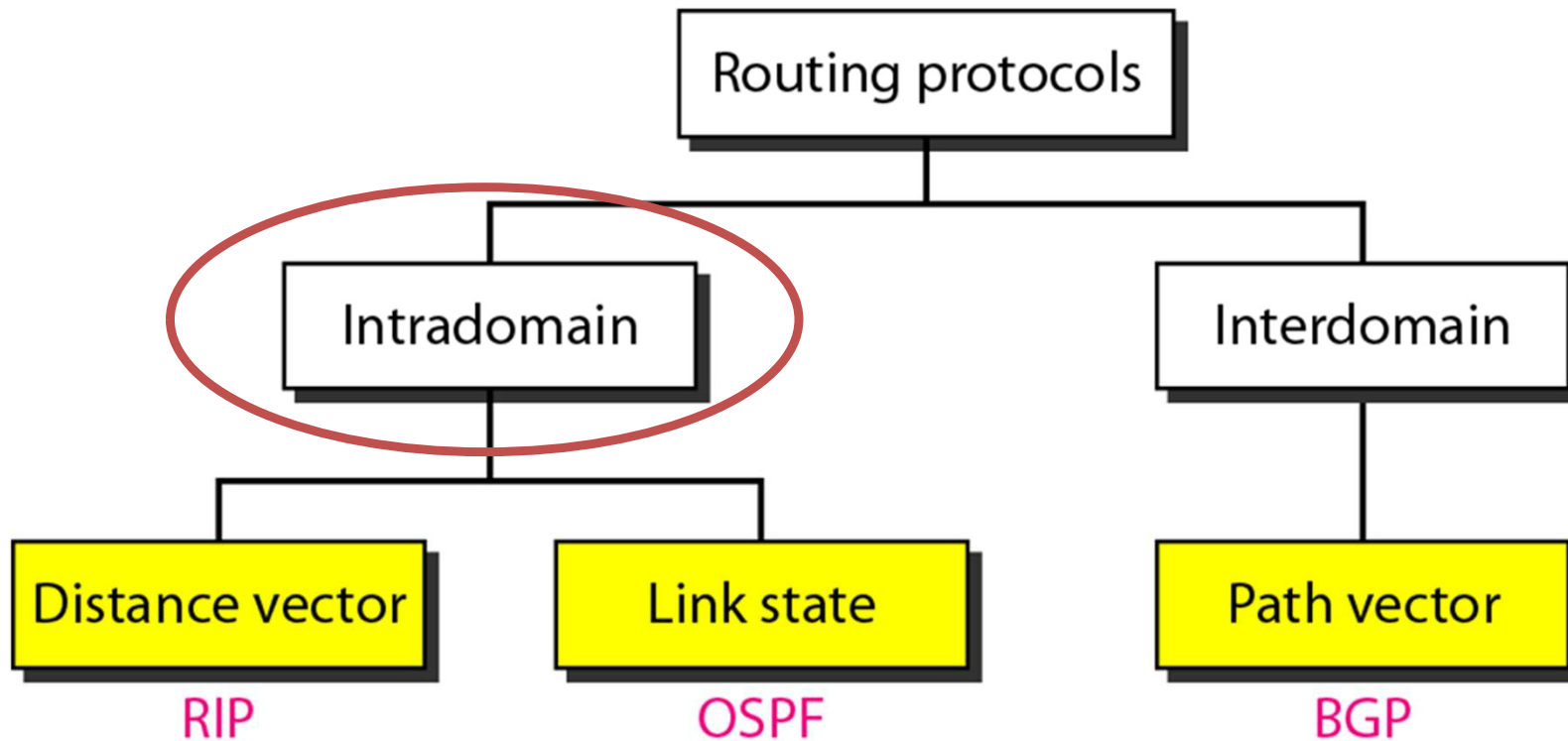
- 선릉 → 사당 → 서울역 → 시청



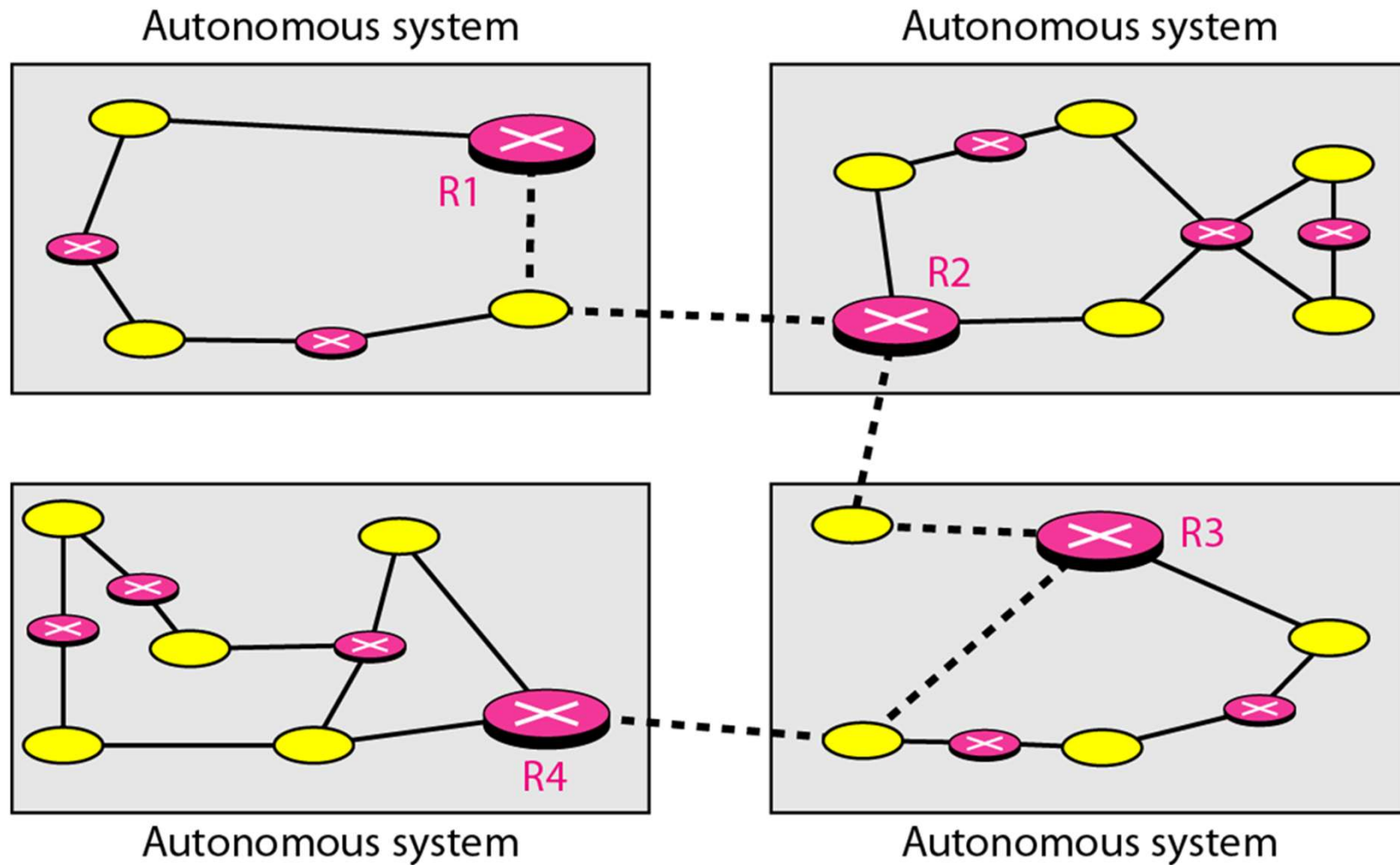
Routing

- How should we decide our path?
 - The fastest
 - The cheapest
 - Possibility of getting a seat
 - Shortest walking distance
 - etc.
- If we choose “the fastest path”, what should we consider?
 - Number of stations
 - Time duration between stations
 - Time for transfer
 - ...

Routing protocols taxonomy

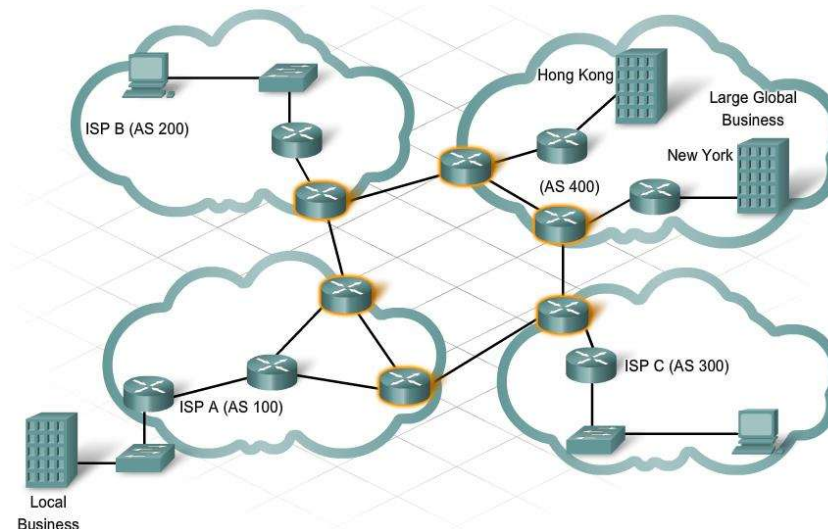


Intra-domain vs. Inter-domain



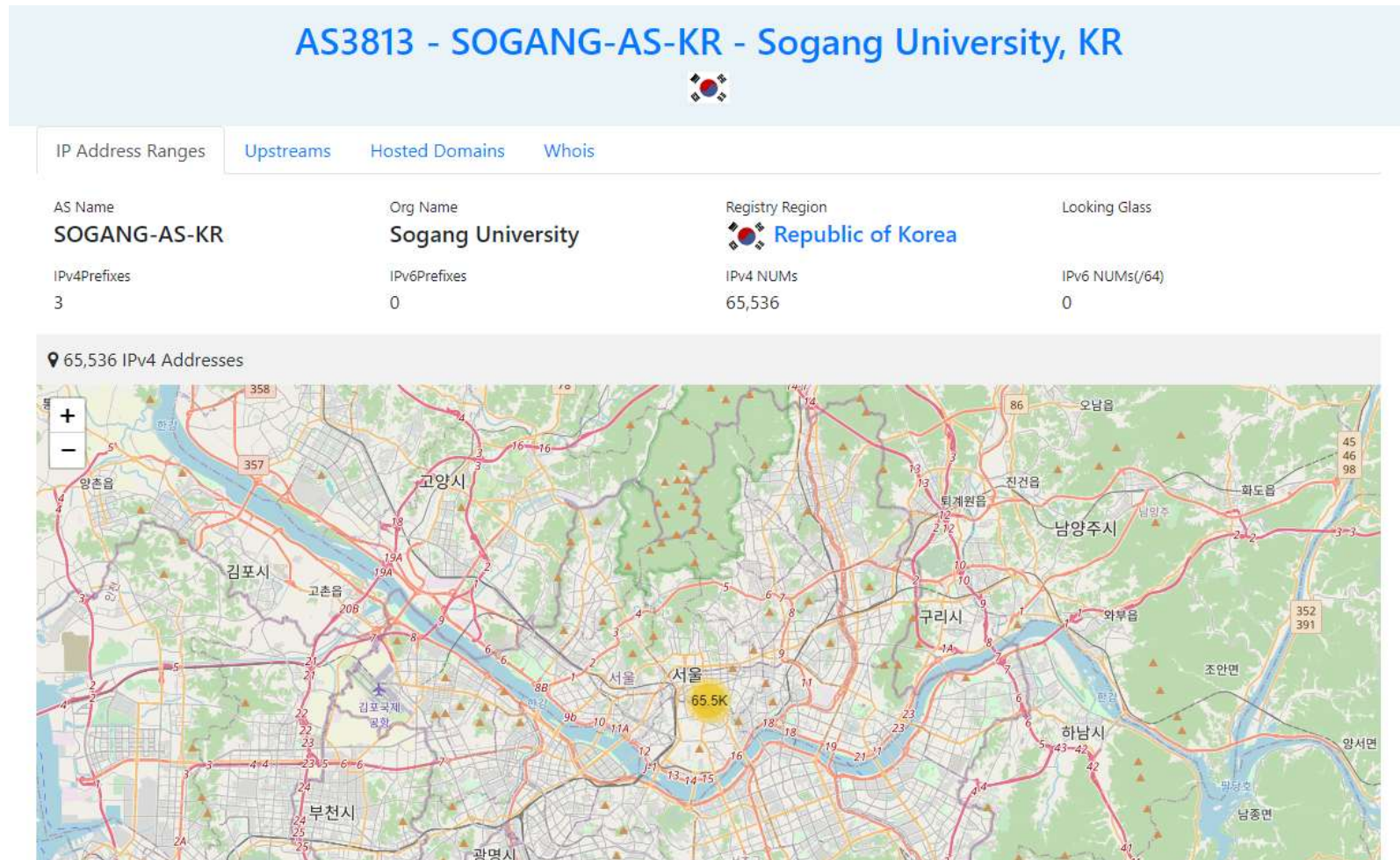
Autonomous Systems (AS)

- Internet: Network of Networks
- Autonomous System: a large network or a group of networks that has a unified routing policy
- Controlled by ISP or an organization
- Each AS has a unique ASN (AS number)
- approximately 120,000 AS in the world (1,187 in Korea)



Autonomous Systems (AS)

- AS3813 - Sogang University



Intra vs. Inter-domain Routing

- Intra-domain (Intra-AS) routing
 - routing among hosts, routers in same AS ("network")
 - all routers in AS must run the same intra-domain protocol
 - routers in different AS can run different intra-domain routing protocol
 - gateway router: at "edge" of its own AS, has link(s) to router(s) in other AS'es
- Inter-domain (Inter-AS) routing
 - routing among AS'es
 - gateways perform inter-domain routing (as well as intra-domain routing)

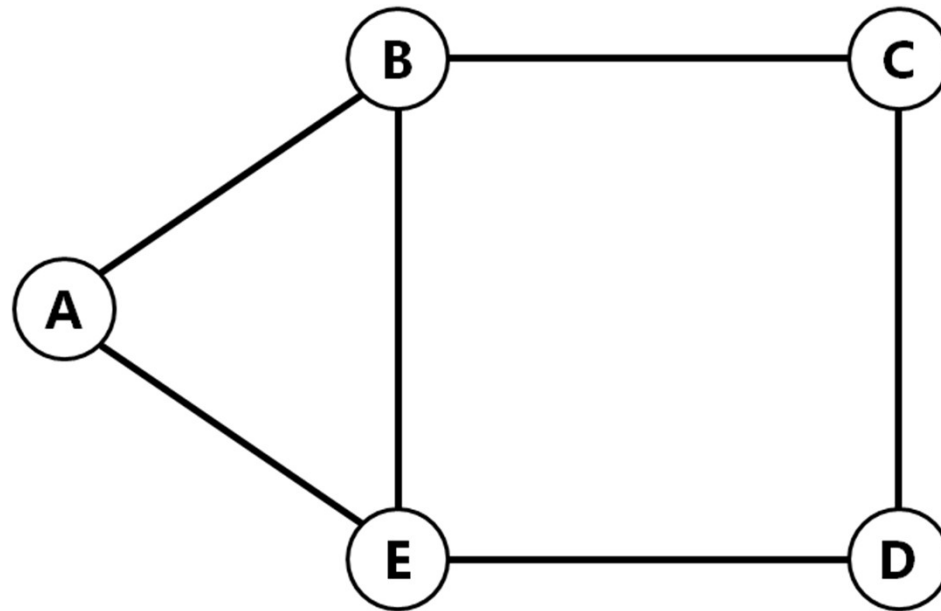
Routing Algorithms & Protocols

Distance vector routing

- Record the next node for each destination
- Exchange tables with neighbors, and update my table using exchanged information
- Information stored at routing tables
 - (destination, distance, next node) → distance vector

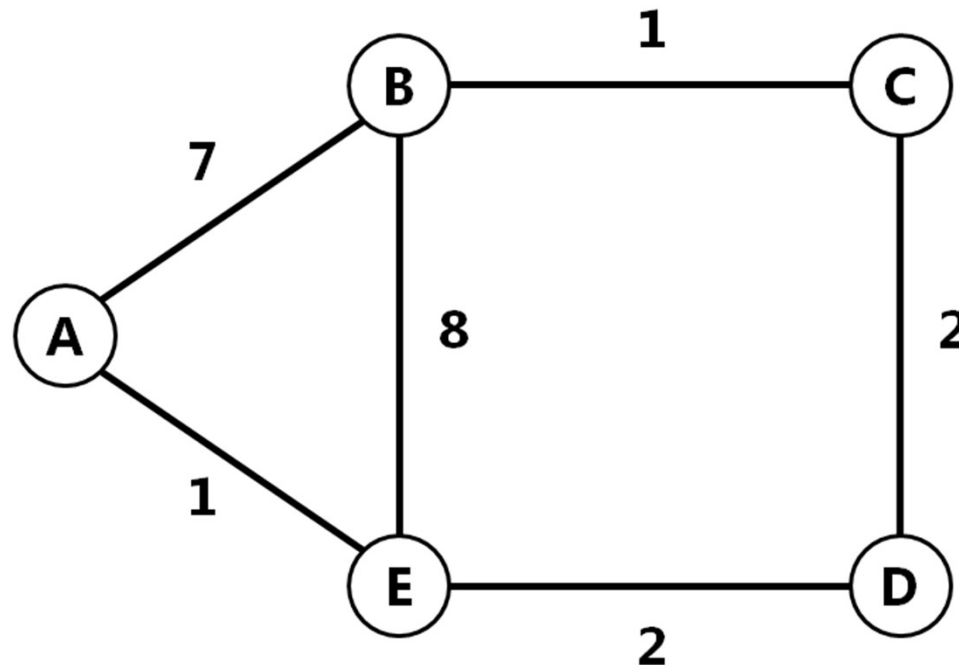
Distance vector routing

- What is the shortest path from A to C?



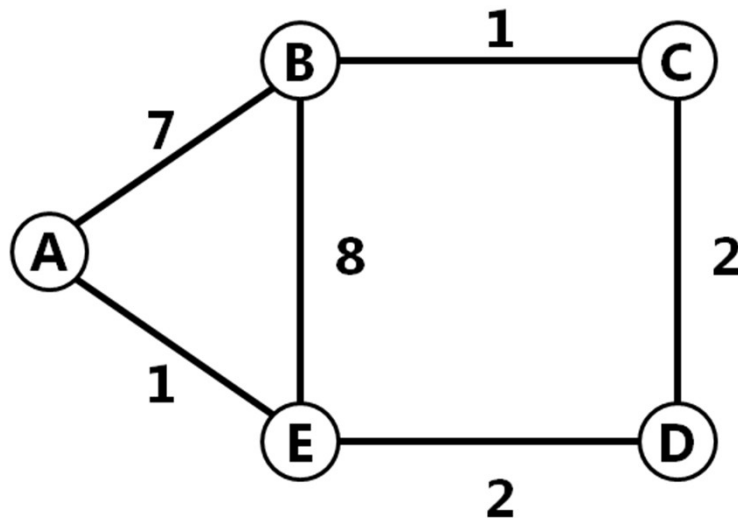
Distance vector routing

- First we need to know the “distance” of each link
 - Distance can be physical distance, but it can also mean other things (e.g. cost)



Distance vector routing

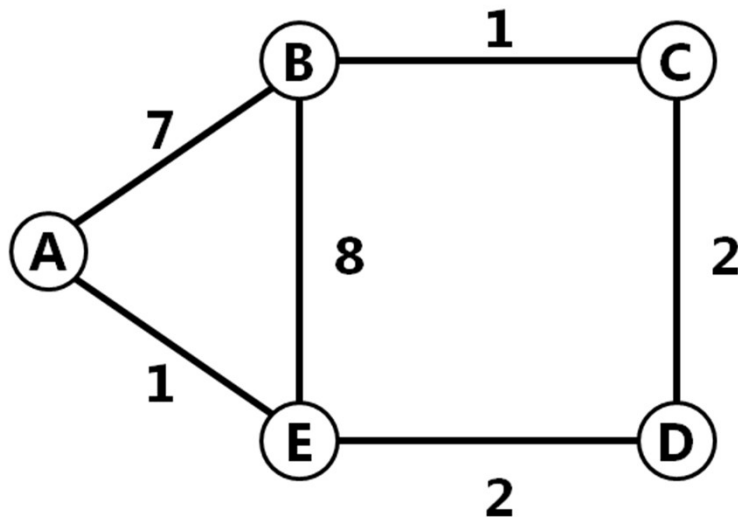
- In the routing table, a “distance vector” is stored for each destination
 - distance vector = (destination, distance, next node)



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	-	-
D	-	-
E	1	E

Distance vector routing

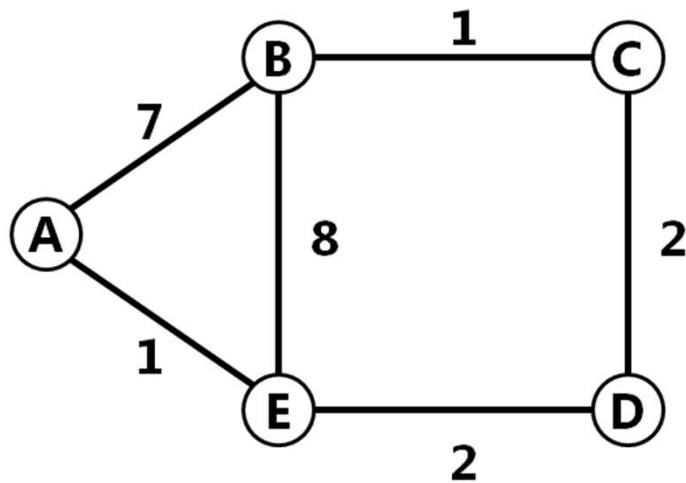
- Initially, we know the paths to the direct neighbors
 - A to B: distance 7: next node B
 - A to E: distance 1: next node E
- Store this information in the routing table



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	-	-
D	-	-
E	1	E

Distance vector routing

- Initial state of routing tables for all routers



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	-	-
D	-	-
E	1	E

B의 Table		
목적지	거리	다음
A	7	A
B	0	-
C	1	C
D	-	-
E	8	E

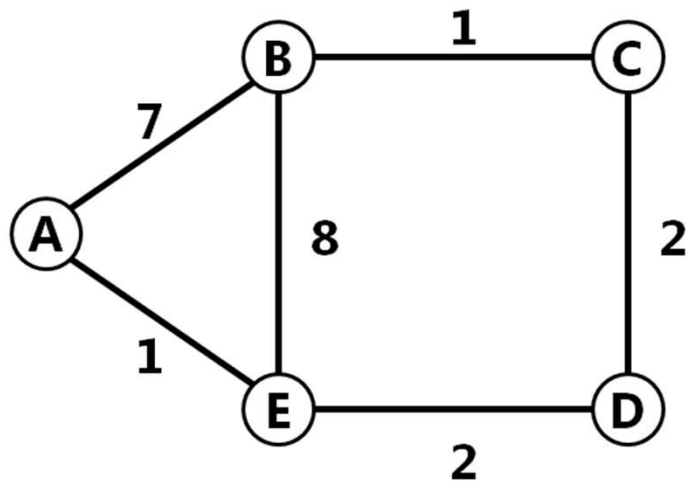
C의 Table		
목적지	거리	다음
A	-	-
B	1	B
C	0	-
D	2	D
E	-	-

D의 Table		
목적지	거리	다음
A	-	-
B	-	-
C	2	C
D	0	-
E	2	E

E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	-	-
D	2	D
E	0	-

Distance vector routing

- Now, nodes exchange tables with direct neighbors
 - Each node sends its routing table to all its direct neighbors



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	-	-
D	-	-
E	1	E

B의 Table		
목적지	거리	다음
A	7	A
B	0	-
C	1	C
D	-	-
E	8	E

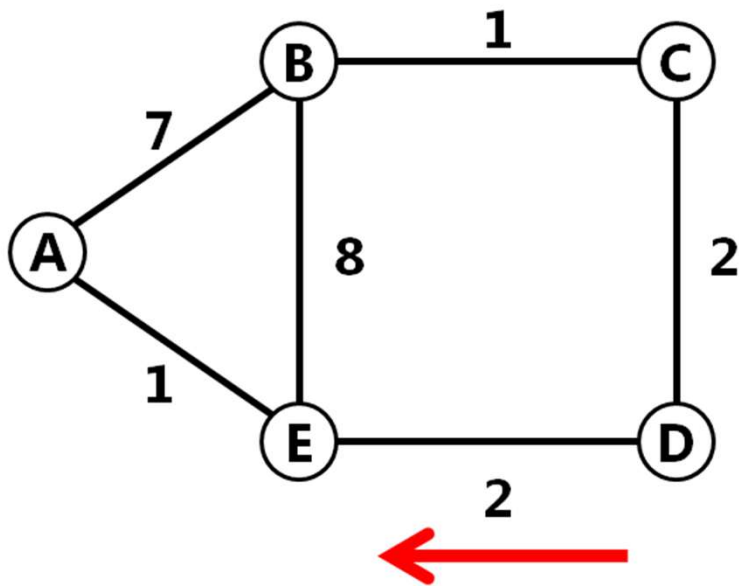
C의 Table		
목적지	거리	다음
A	-	-
B	1	B
C	0	-
D	2	D
E	-	-

D의 Table		
목적지	거리	다음
A	-	-
B	-	-
C	2	C
D	0	-
E	2	E

E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	-	-
D	2	D
E	0	-

Distance vector routing

- Node D sends its table to node E
 - Information sent: (destination, distance)
 - Next node should be the sending node itself



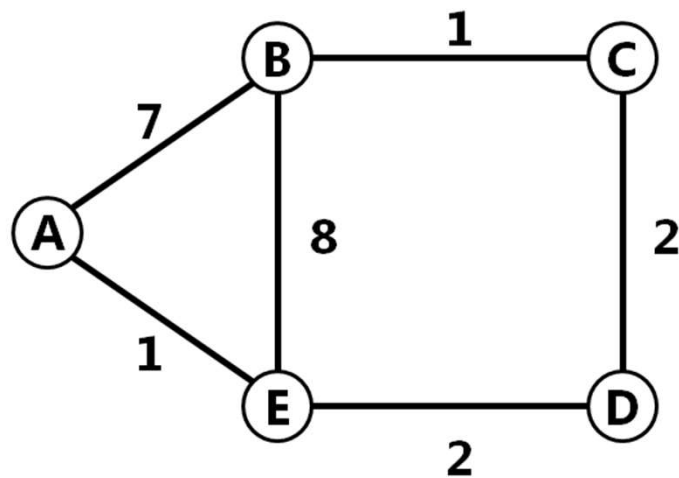
D의 Table		
목적지	거리	다음
A	-	-
B	-	-
C	2	C
D	0	-
E	2	E

E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	-	-
D	2	D
E	0	-

C	2
D	0
E	2

Distance vector routing

- On receiving table from D, node E updates its routing table
 - E didn't know the path to C, but now it knows that it can reach C through node D, and the distance will be 4.
 - $2 (E \rightarrow D) + 2 (D \rightarrow C)$



D의 Table		
목적지	거리	다음
A	-	-
B	-	-
C	2	C
D	0	-
E	2	E

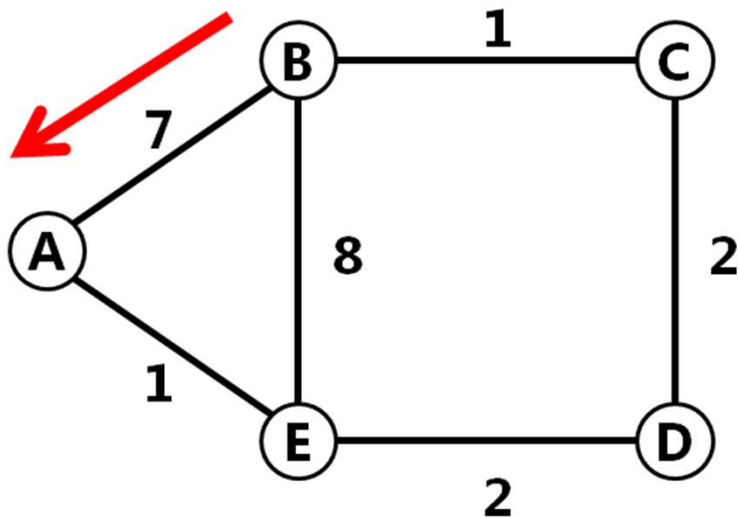
E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	4	D
D	2	D
E	0	-

C	2
D	0
E	2

Distance vector routing

- Node B sends its table to node A

A	7
B	0
C	1
E	8



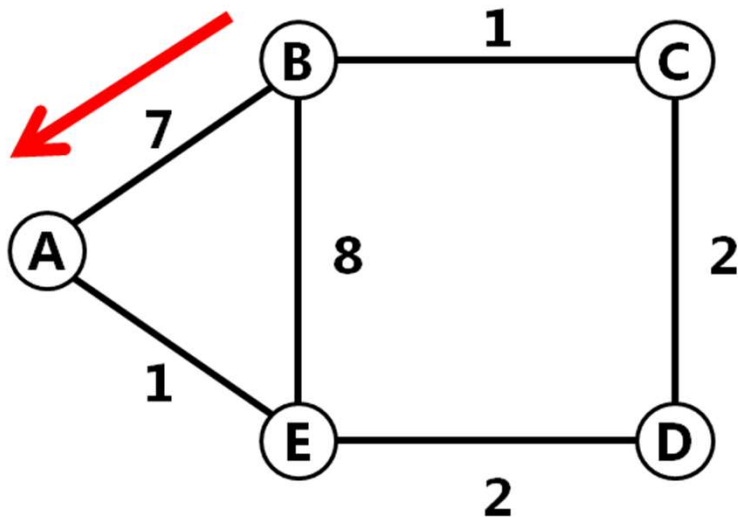
A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	-	-
D	-	-
E	1	E

B의 Table		
목적지	거리	다음
A	7	A
B	0	-
C	1	C
D	-	-
E	8	E

Distance vector routing

- A updates its routing table
 - A knows it can reach C through B, with distance 8.

A	7
B	0
C	1
E	8

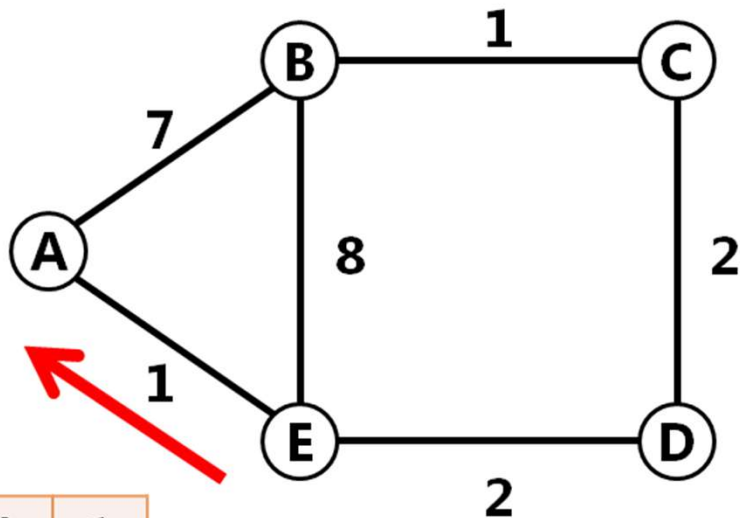


A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	8	B
D	-	-
E	1	E

B의 Table		
목적지	거리	다음
A	7	A
B	0	-
C	1	C
D	-	-
E	8	E

Distance vector routing

- Node E sends its table to node A



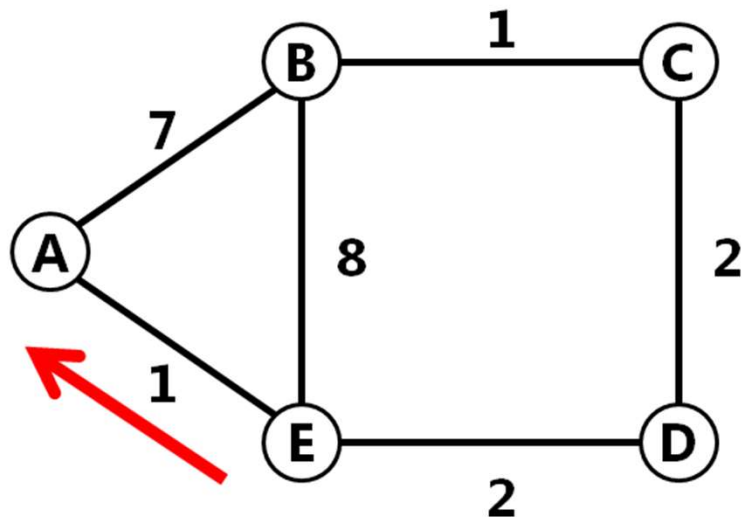
A	1
B	8
C	4
D	2
E	0

A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	8	B
D	-	-
E	1	E

E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	4	D
D	2	D
E	0	-

Distance vector routing

- Node A updates its table
 - Since distance from E to C is 4, E can reach C with distance 5.
 - Now, node A knows a path to C with distance 8.
 - Since the path through E is shorter, node A updates its path to C to go through E.
 - If the current path is shorter or equal, it is not updated.



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	5	E
D	3	E
E	1	E

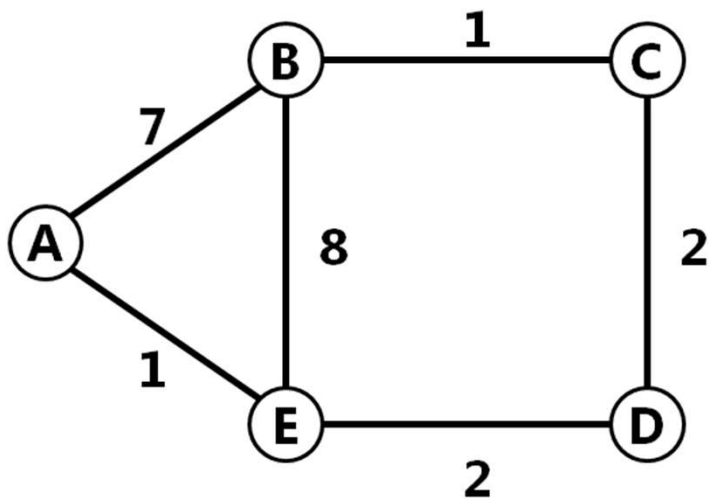
E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	4	D
D	2	D
E	0	-

Distance vector routing

- All nodes exchange tables
 - Send my table to all my neighbors
 - If my table is updated, send my table again
 - Also, nodes send its table periodically
 - node state or link distance might have changed
- After all exchanges have been finished...

Distance vector routing

- The final state of routing tables



A의 Table		
목적지	거리	다음
A	0	-
B	6	E
C	5	E
D	3	E
E	1	E

B의 Table		
목적지	거리	다음
A	6	C
B	0	-
C	1	C
D	3	C
E	5	C

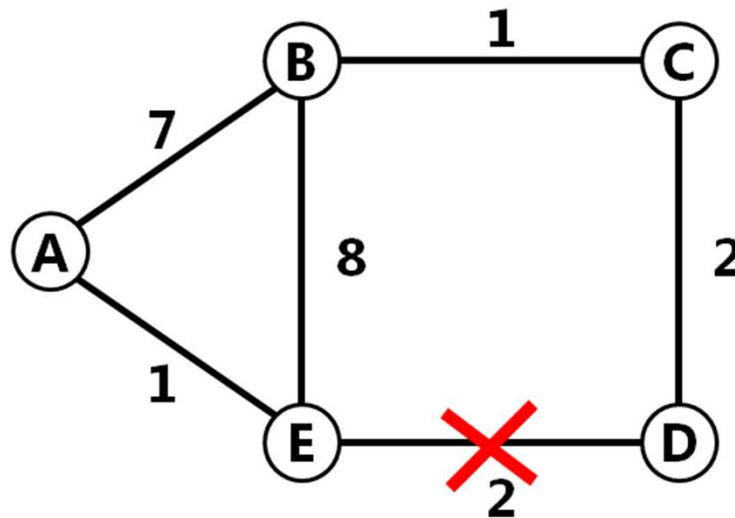
C의 Table		
목적지	거리	다음
A	5	D
B	1	B
C	0	-
D	2	D
E	4	D

D의 Table		
목적지	거리	다음
A	3	E
B	3	C
C	2	C
D	0	-
E	2	E

E의 Table		
목적지	거리	다음
A	1	A
B	5	D
C	4	D
D	2	D
E	0	-

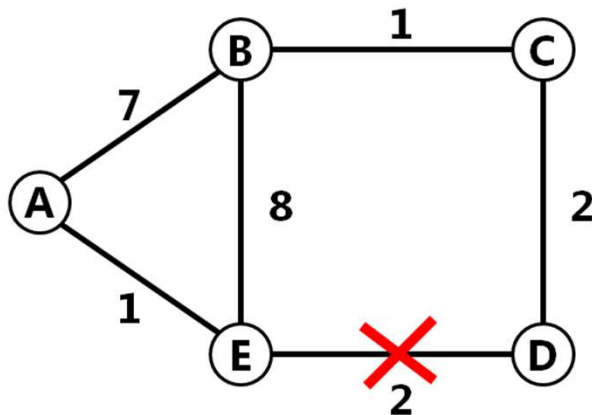
Distance vector routing

- Suppose the link between D and E breaks
 - D and E detect this event



Distance vector routing

- D and E sends the updated routing table to their neighbors, and nodes update their routing table



A의 Table		
목적지	거리	다음
A	0	-
B	7	B
C	8	B
D	10	B
E	1	E

B의 Table		
목적지	거리	다음
A	7	A
B	0	-
C	1	C
D	3	C
E	8	E

C의 Table		
목적지	거리	다음
A	8	B
B	1	B
C	0	-
D	2	D
E	9	B

D의 Table		
목적지	거리	다음
A	10	C
B	3	C
C	2	C
D	0	-
E	11	C

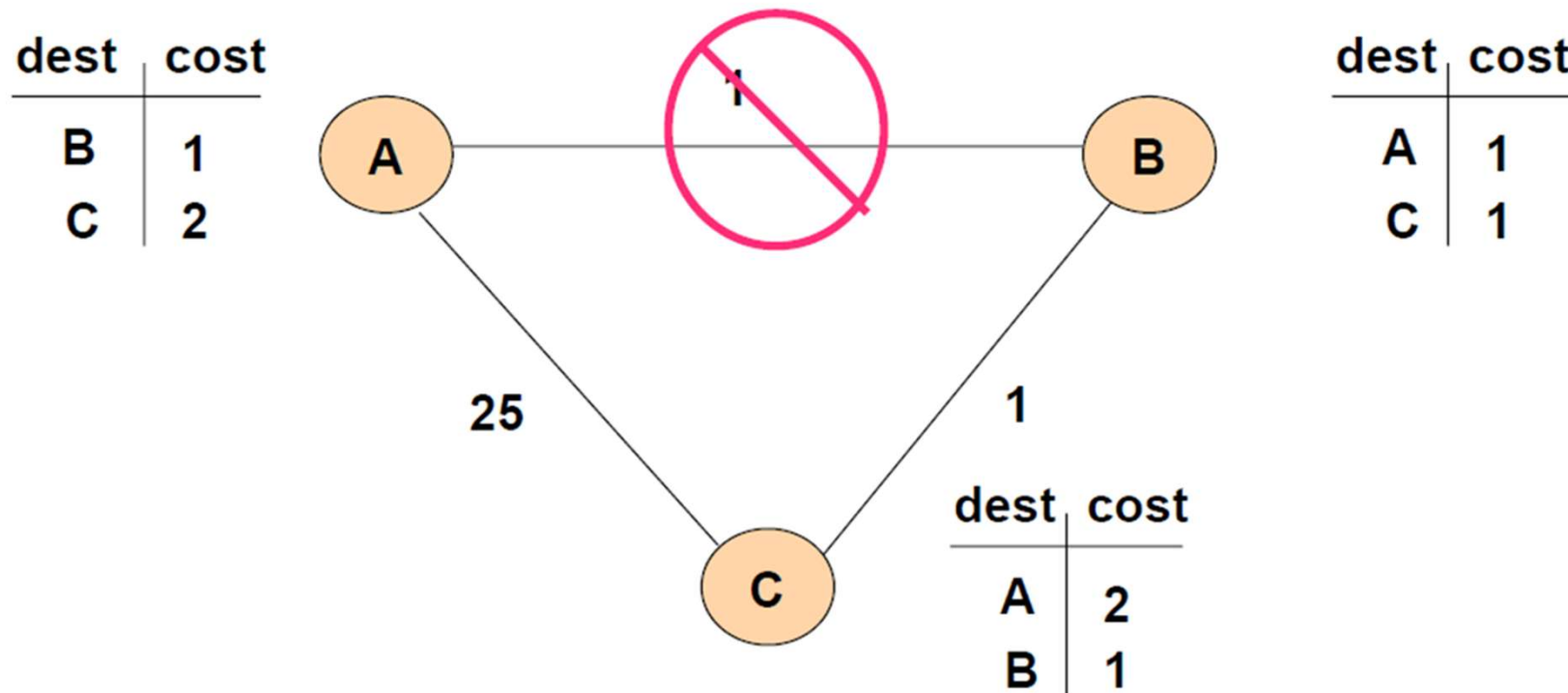
E의 Table		
목적지	거리	다음
A	1	A
B	8	B
C	9	B
D	11	B
E	0	-

Distance vector routing

- Nodes periodically exchange routing tables with direct neighbors
- If there is a change in the network, the routing table will be updated to adopt the change
- Each route information has a timer. If the route is not updated for a certain duration, it is invalidated.
 - Suppose A has a route to C via B. If node A does not get periodic updates from B, the route to C will be removed after some time.

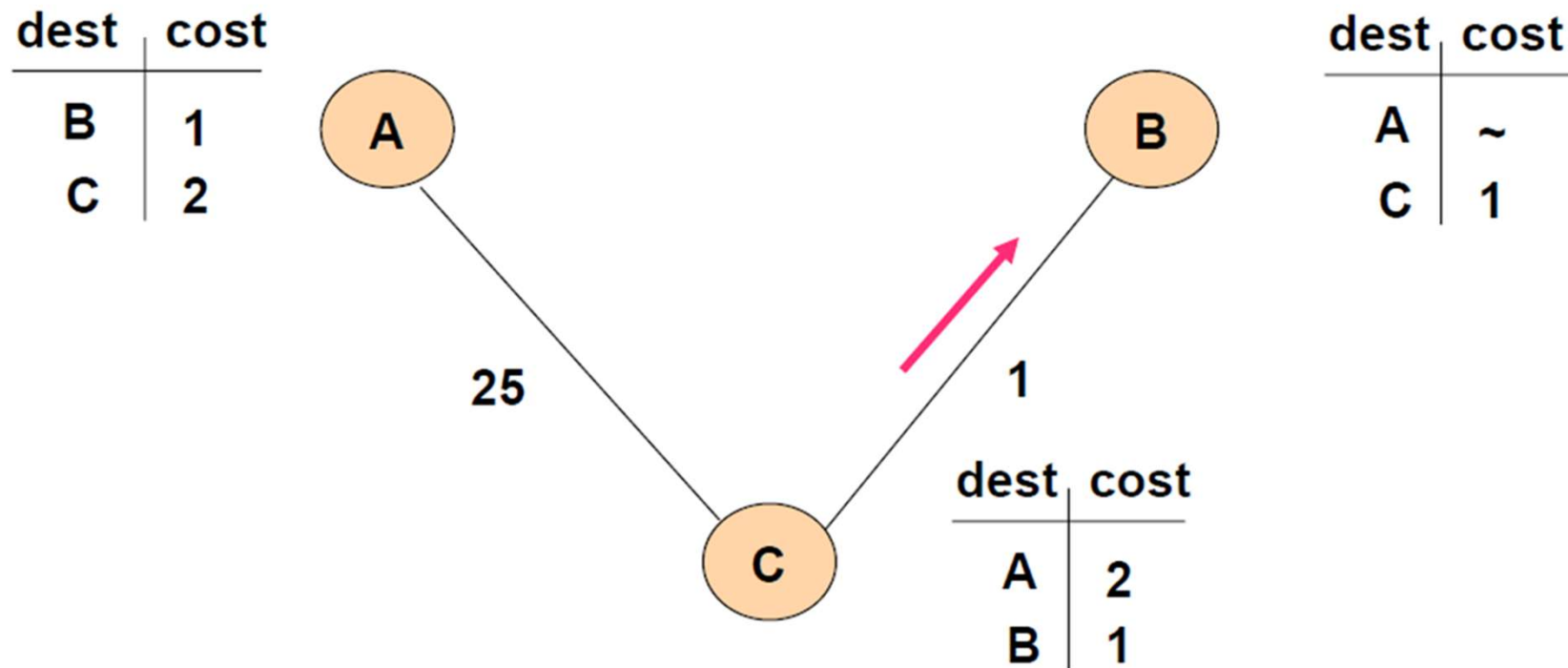
Count-to-infinity problem

- A problem in distance vector routing
- Consider the following network. After route table is finalized, the link between A and B breaks



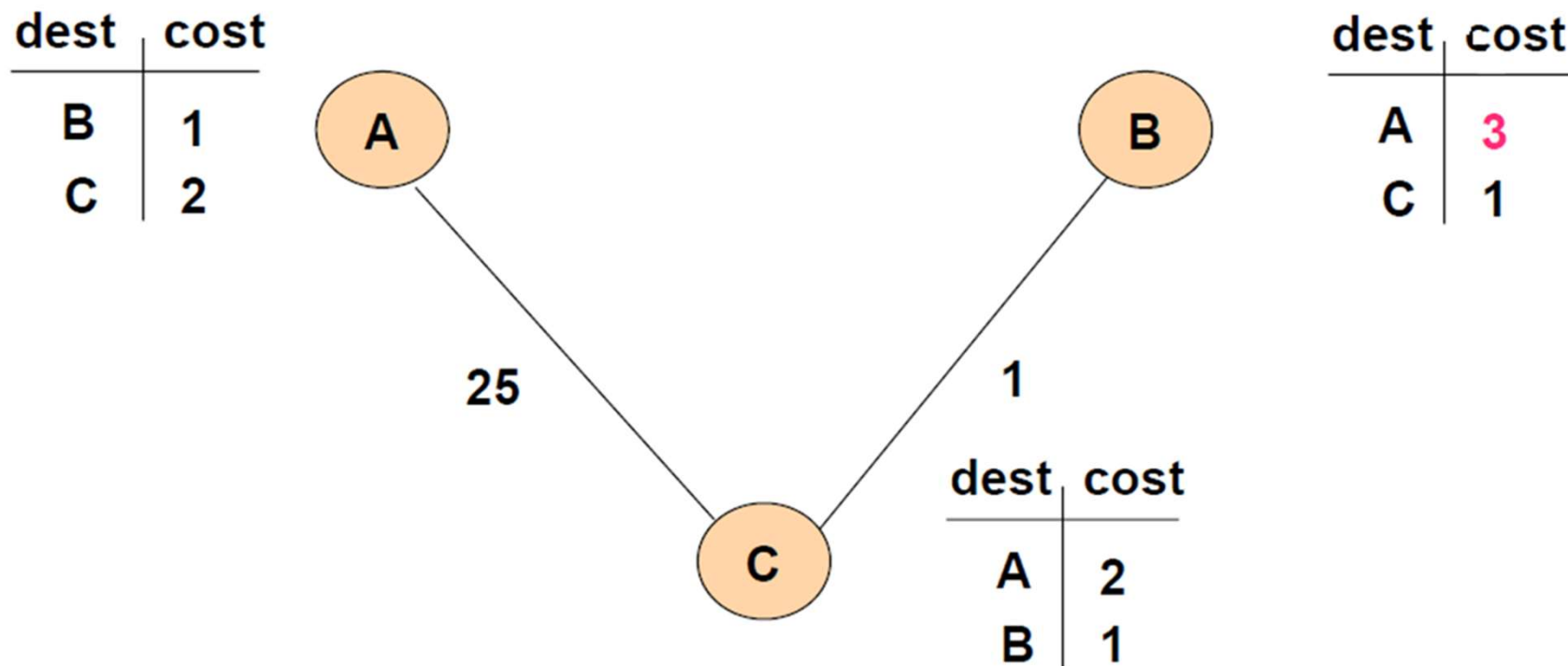
Count-to-infinity problem

- B does not know how to reach A
- C sends its routing table to B



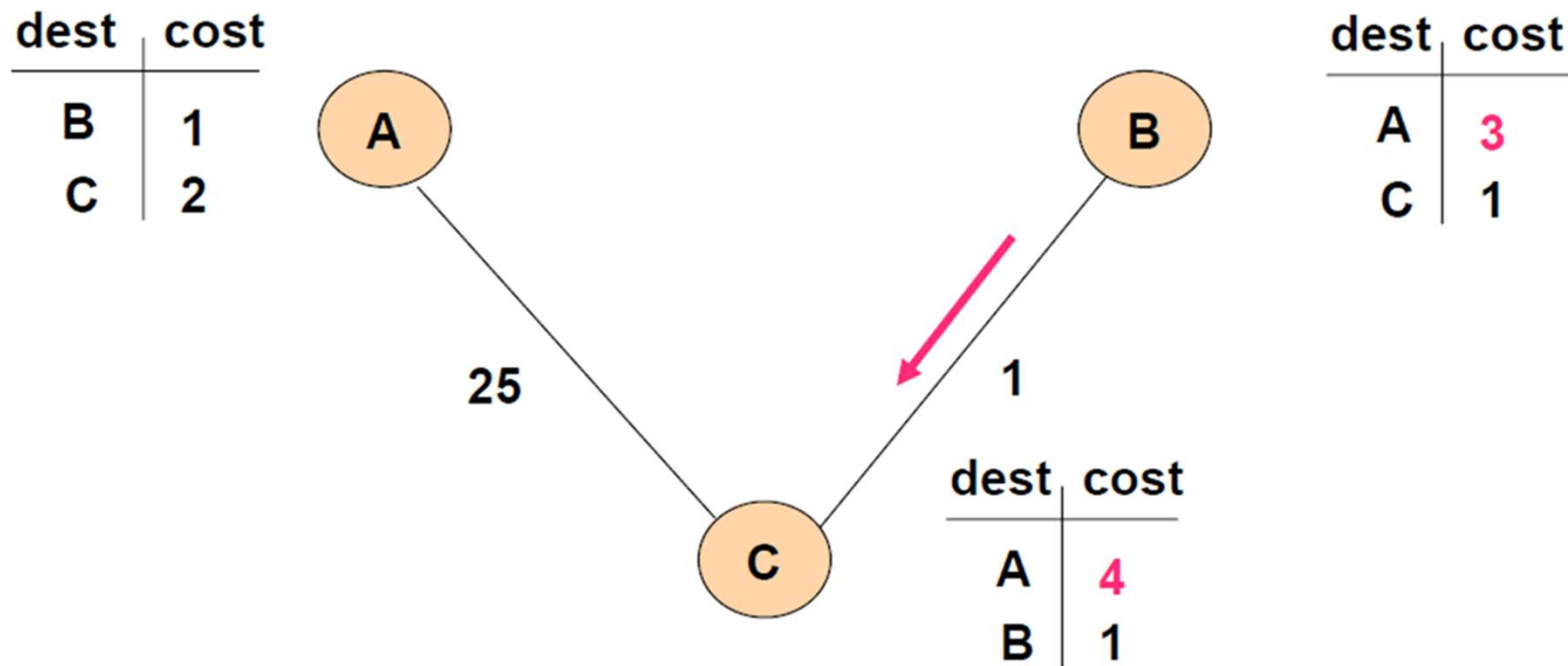
Count-to-infinity problem

- B thinks it can reach A via C with distance 3.
 - However, this path does not exist. ($B \rightarrow C \rightarrow B \rightarrow A$)



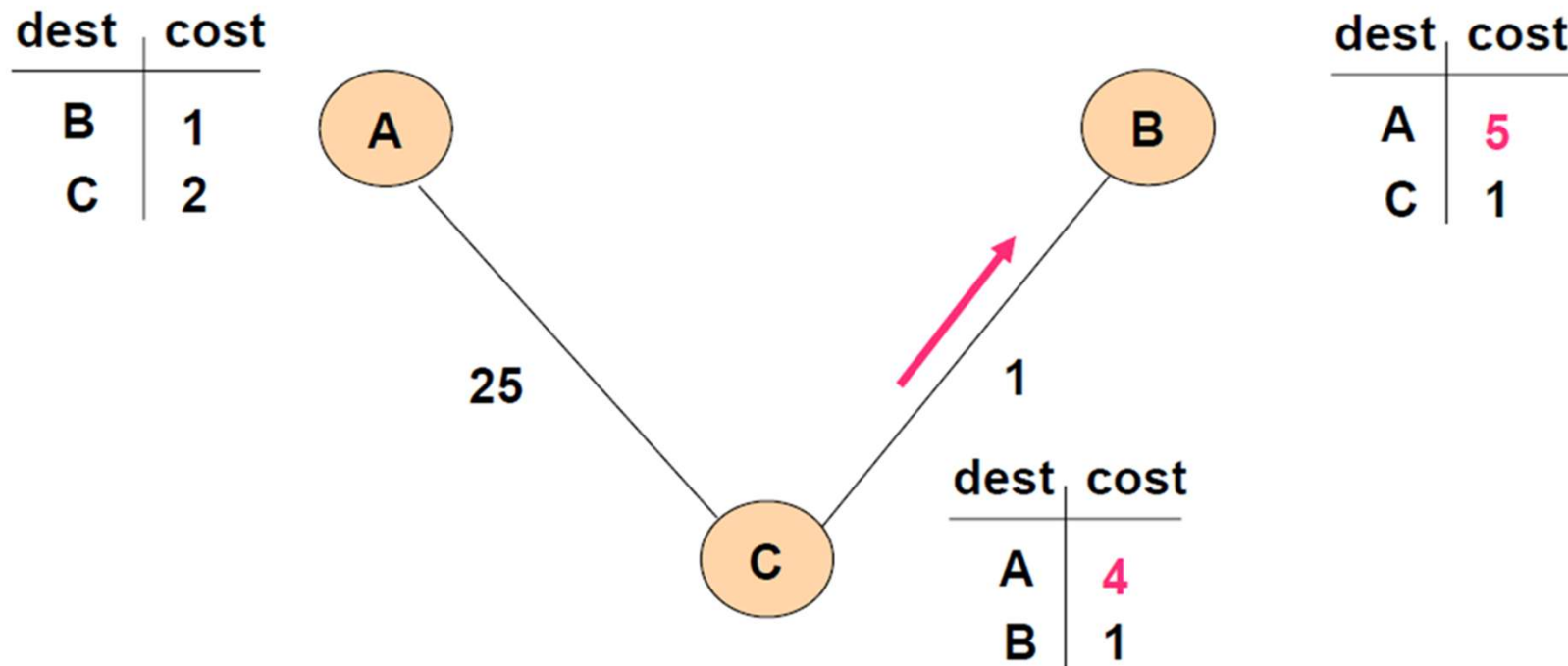
Count-to-infinity problem

- Now B sends its table to C and C updates its table
 - The distance to C has changed.



Count-to-infinity problem

- C sends its table to B
- B updates its table

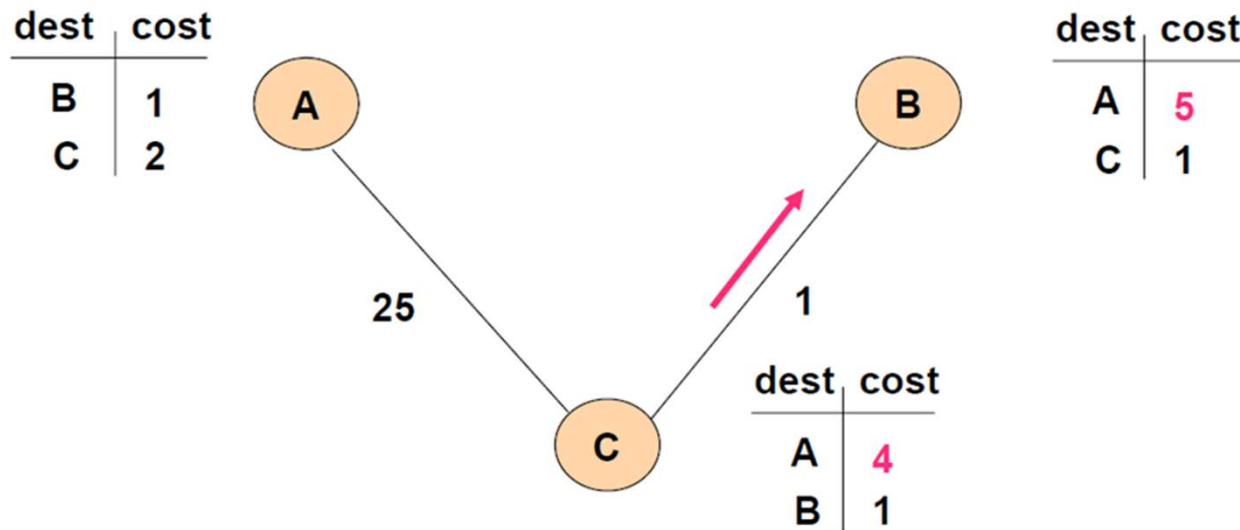


Count-to-infinity problem

- C will eventually change its path to A (direct path), but too much time is needed until C changes its path.
- Also, too much route information is exchanged
 - waste of network resource

Count-to-infinity problem

- Cause of the problem
 - When B receives route information from C, B does not know whether the route goes through itself.
 - This problem arises because distance vector only records the next node (not the whole path)

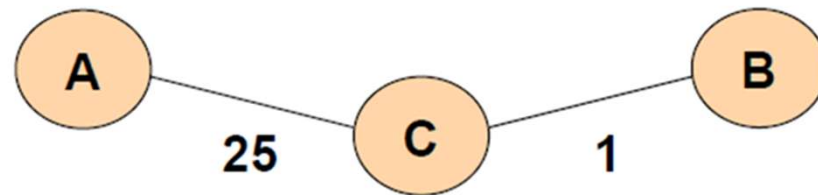


Count-to-infinity problem

- Solutions to problem
 - split horizon
 - split horizon with poison reverse
 - path vector routing

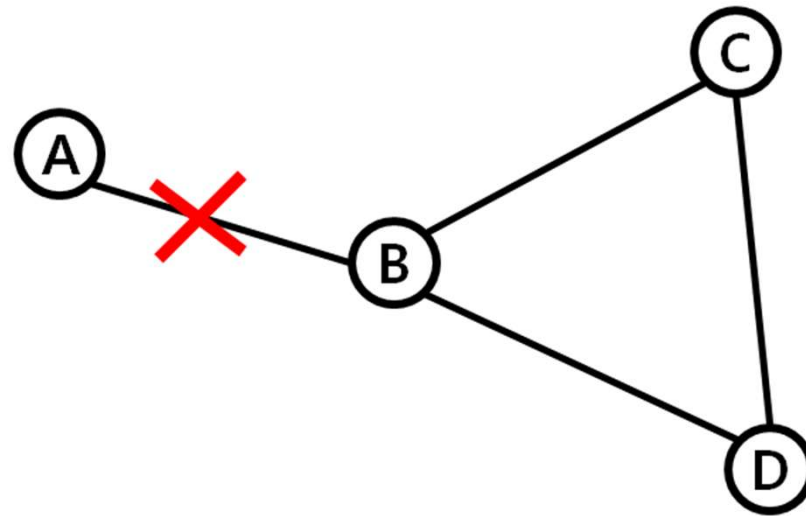
Count-to-infinity problem

- Split horizon
 - When C sends its route table to B, do not include routes that go through B
- Split horizon with poison reverse
 - When C sends its route table to B, include routes that go through B, but make their distance to be infinity



Count-to-infinity problem

- Split horizon with poison reverse: limitation
 - 3-hop loop cannot be avoided

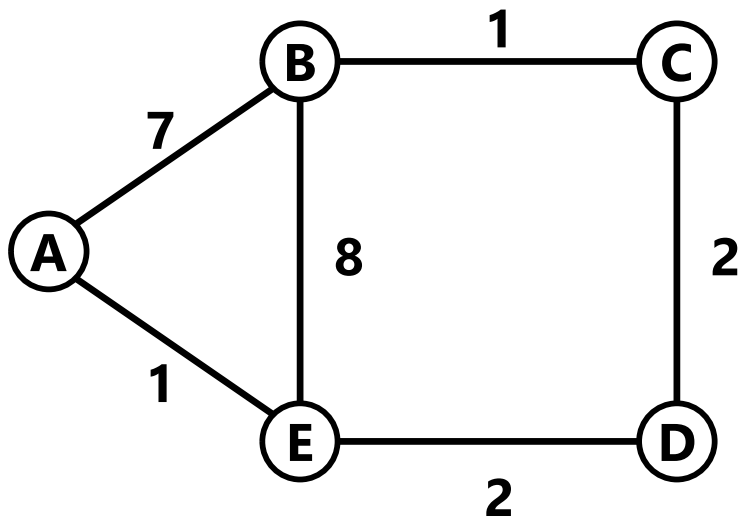


Count-to-infinity problem

- Path vector routing
 - record the whole path (not just the next node)
 - loop can be detected as soon as the table is received
 - used in BGP (Border Gateway Protocol)

Path vector routing

- record distance and the whole path



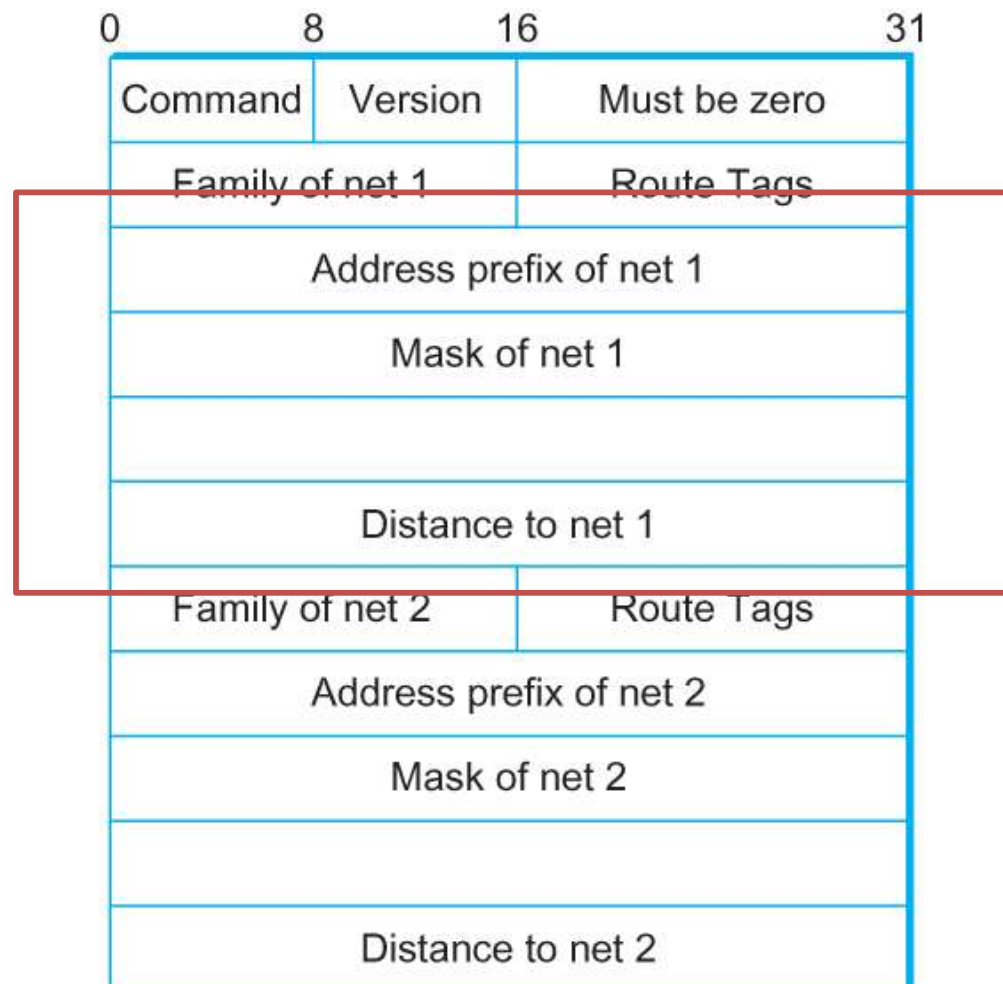
A의 Table		
목적지	거리	경로
A	0	-
B	6	E D C B
C	5	E D C
D	3	E D
E	1	E

Routing Information Protocol (RIP)

- A simple distance-vector routing protocol
 - Historically widely used in small networks
 - Today mostly used in lab/educational settings
- Each router sends advertisement messages
 - Every 30 seconds, or
 - If routing table is updated

Routing Information Protocol (RIP)

- RIPv2 Packet Format



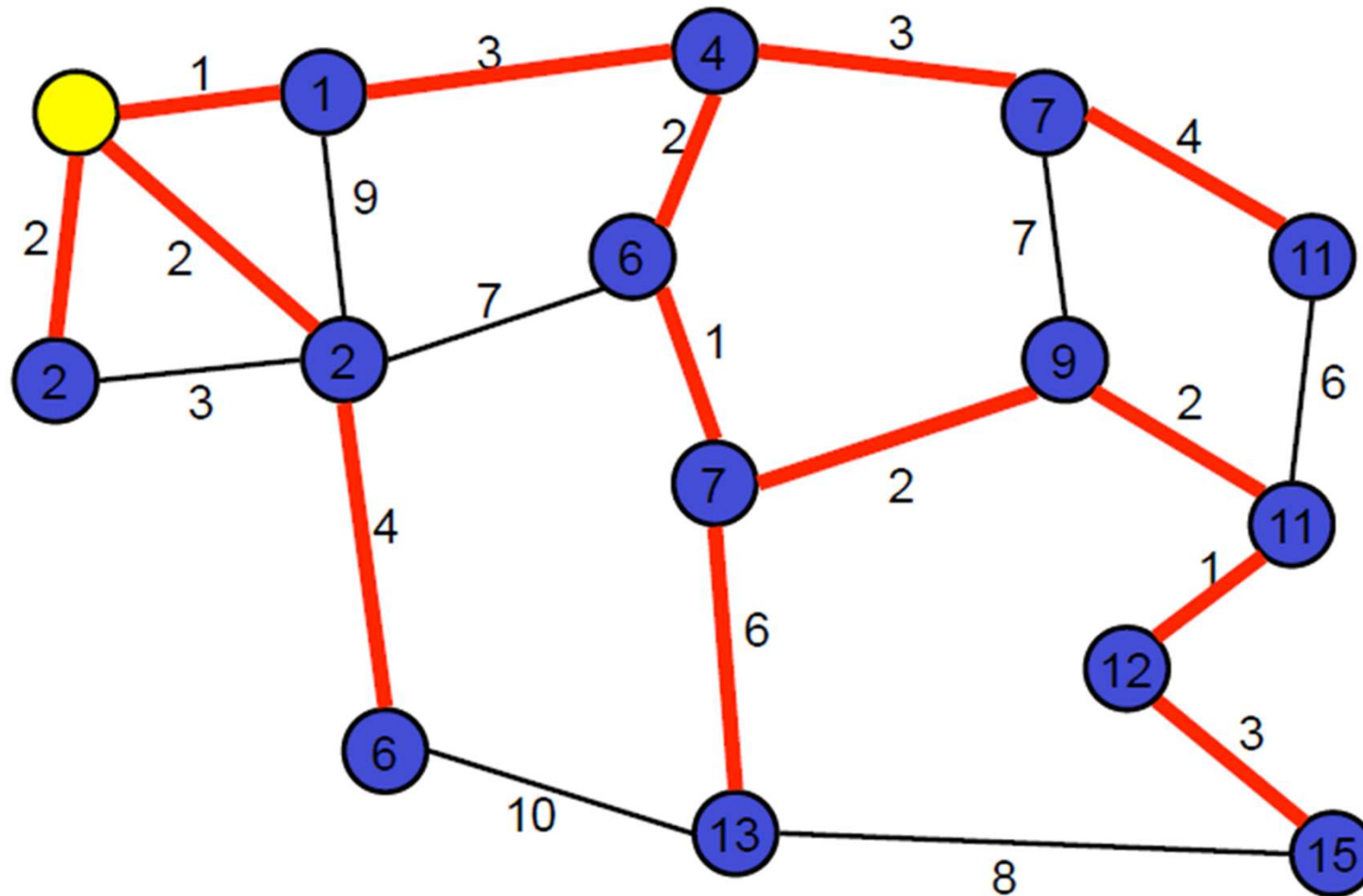
Link-state routing

- Send my "link state" information to all nodes in the network
 - Link state packet (LSP)
- Link state packet
 - ID of the sender
 - (ID, distance) for each direct neighbor of the sender
 - Sequence number (to find out if it is the newest info)
- Once a node receives LSP from other nodes, it uses the Dijkstra algorithm to find out the shortest paths

Dijkstra algorithm

- Input
 - A graph with edge weights (distance)
- Output
 - Shortest paths from one node to all others

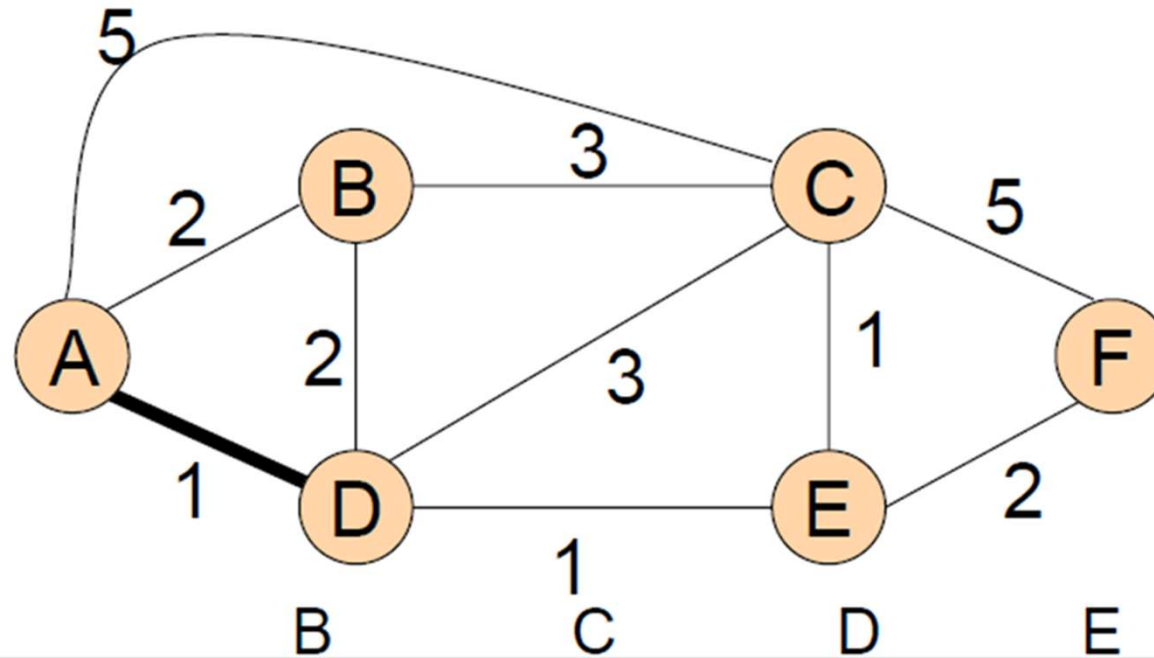
Dijkstra algorithm



Dijkstra algorithm

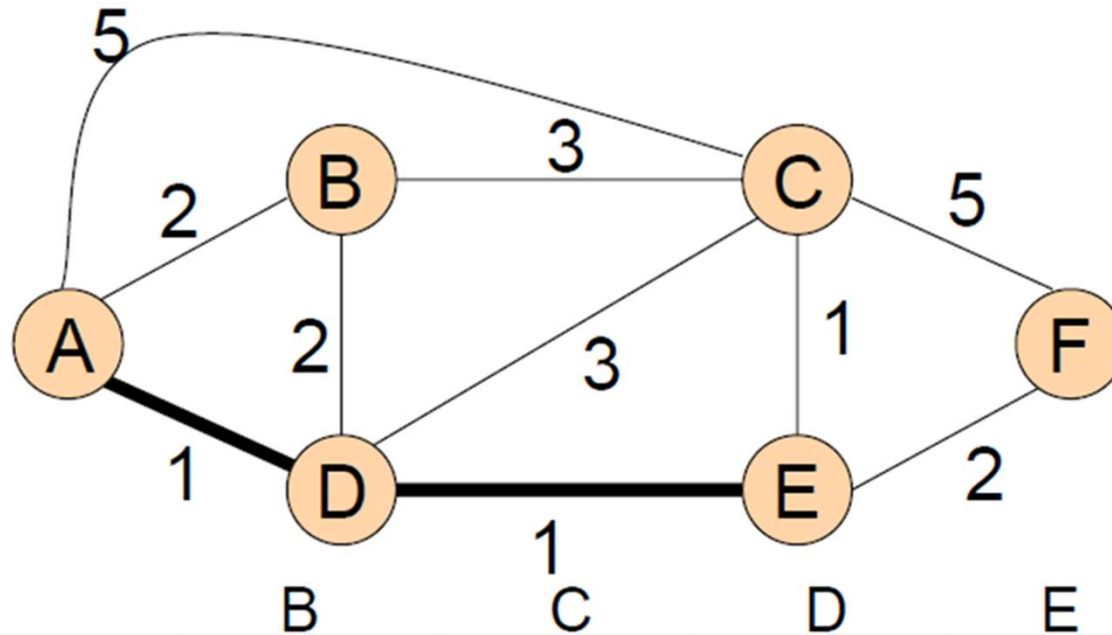
- Initially $\mathbf{C} = \{\text{source}\}$
- Loop $N-1$ times
 - Find node M that is not included in $\mathbf{C}$ and is the closest to the source node
 - Include M in $\mathbf{C}$

Dijkstra algorithm: example



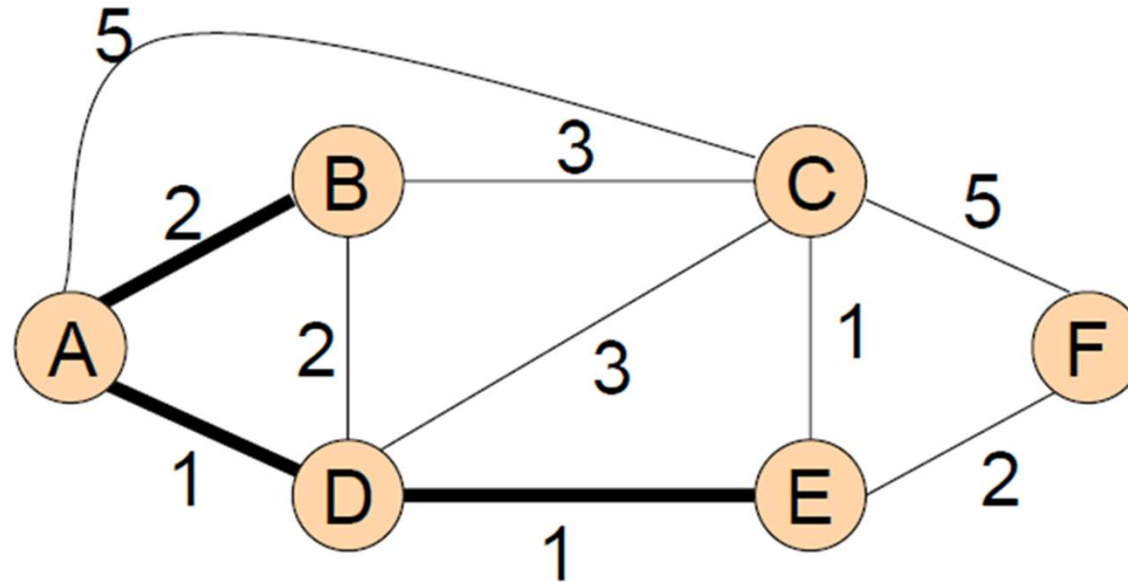
step	SPT	D(b), P(b)	D(c), P(c)	D(d), P(d)	D(e), P(e)	D(f), P(f)
0	A	2, A	5, A	1, A	~	~
1	AD	2, A	4, D		2, D	~

Dijkstra algorithm: example



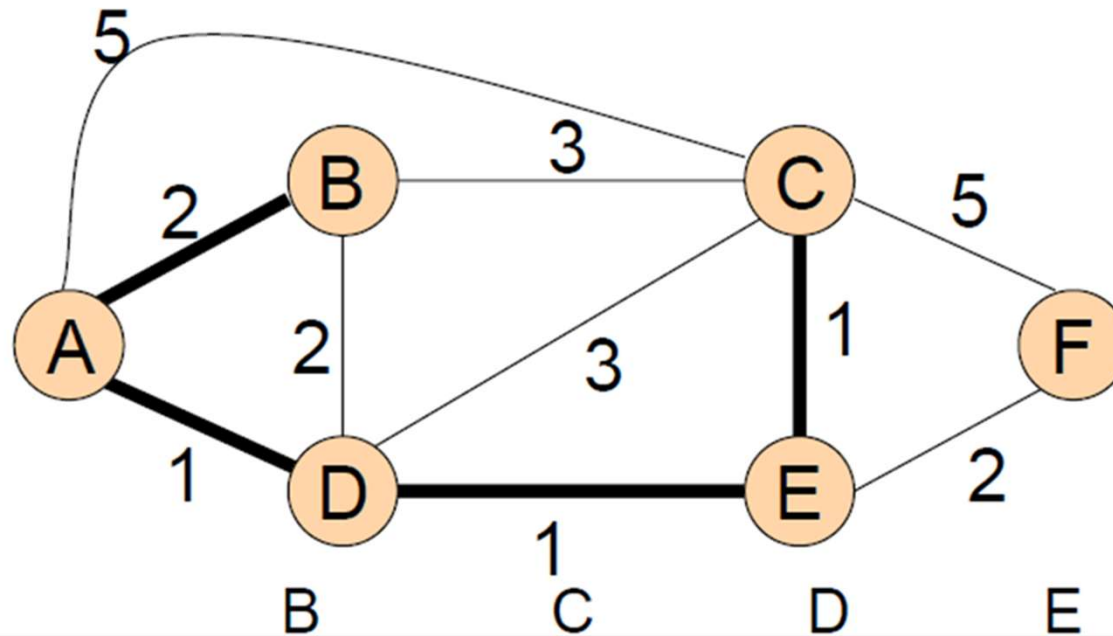
step	SPT	B	C	D	E	F
	SPT	D(b), P(b)	D(c), P(c)	D(d), P(d)	D(e), P(e)	D(f), P(f)
0	A	2, A	5, A	1, A	~	~
1	AD	2, A	4, D		2, D	~
2	ADE	2, A	3, E			4, E

Dijkstra algorithm: example



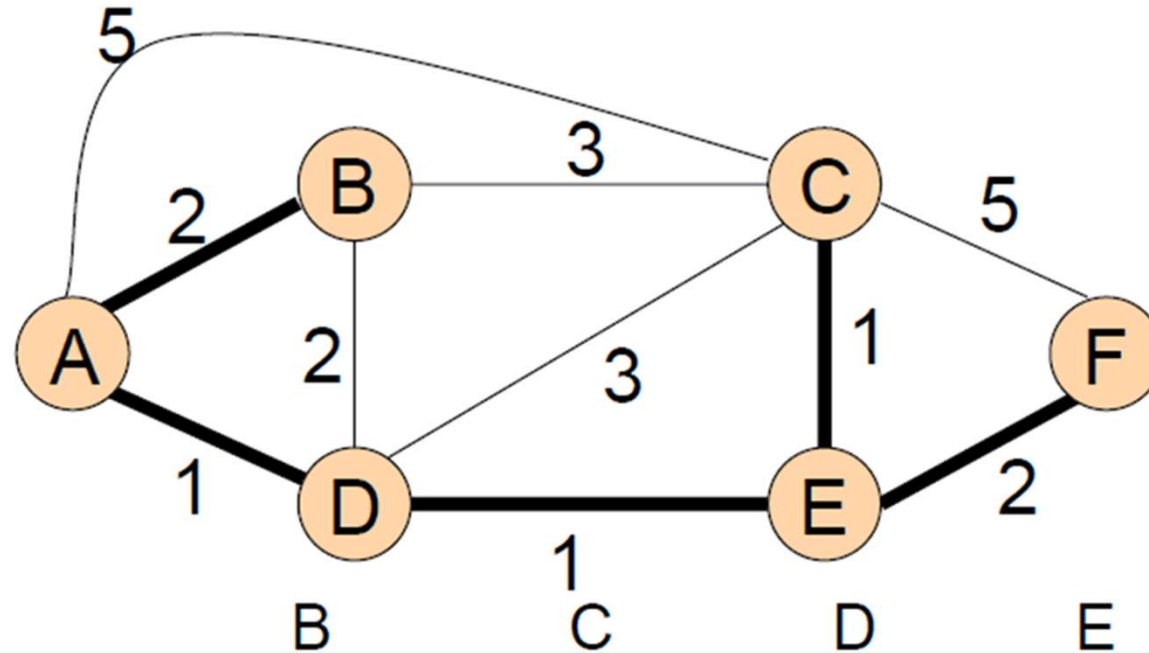
		B	C	D	E	F
step	SPT	D(b), P(b)	D(c), P(c)	D(d), P(d)	D(e), P(e)	D(f), P(f)
0	A	2, A	5, A	1, A	~	~
1	AD	2, A	4, D		2, D	~
2	ADE	2, A	3, E			4, E
3	ADEB		3, E			4, E

Dijkstra algorithm: example



step	SPT	B D(b), P(b)	C D(c), P(c)	D D(d), P(d)	E D(e), P(e)	F D(f), P(f)
0	A	2, A	5, A	1, A	~	~
1	AD	2, A	4, D		2, D	~
2	ADE	2, A	3, E			4, E
3	ADEB		3, E			4, E
4	ADEBC					4, E

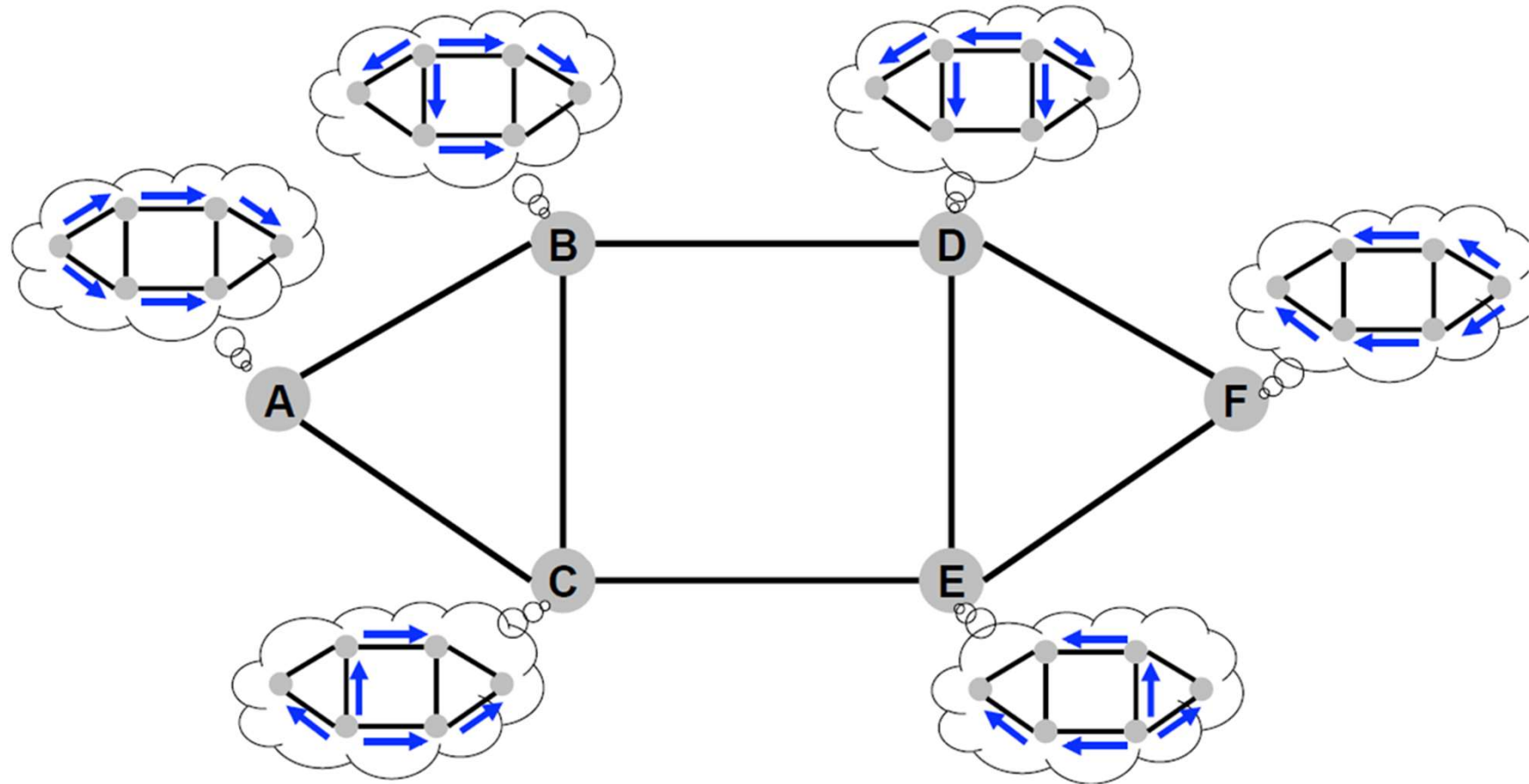
Dijkstra algorithm: example



		B	C	D	E	F
step	SPT	D(b), P(b)	D(c), P(c)	D(d), P(d)	D(e), P(e)	D(f), P(f)
0	A	2, A	5, A	1, A	~	~
1	AD	2, A	4, D		2, D	~
2	ADE	2, A	3, E			4, E
3	ADEB		3, E			4, E
4	ADEBC					4, E

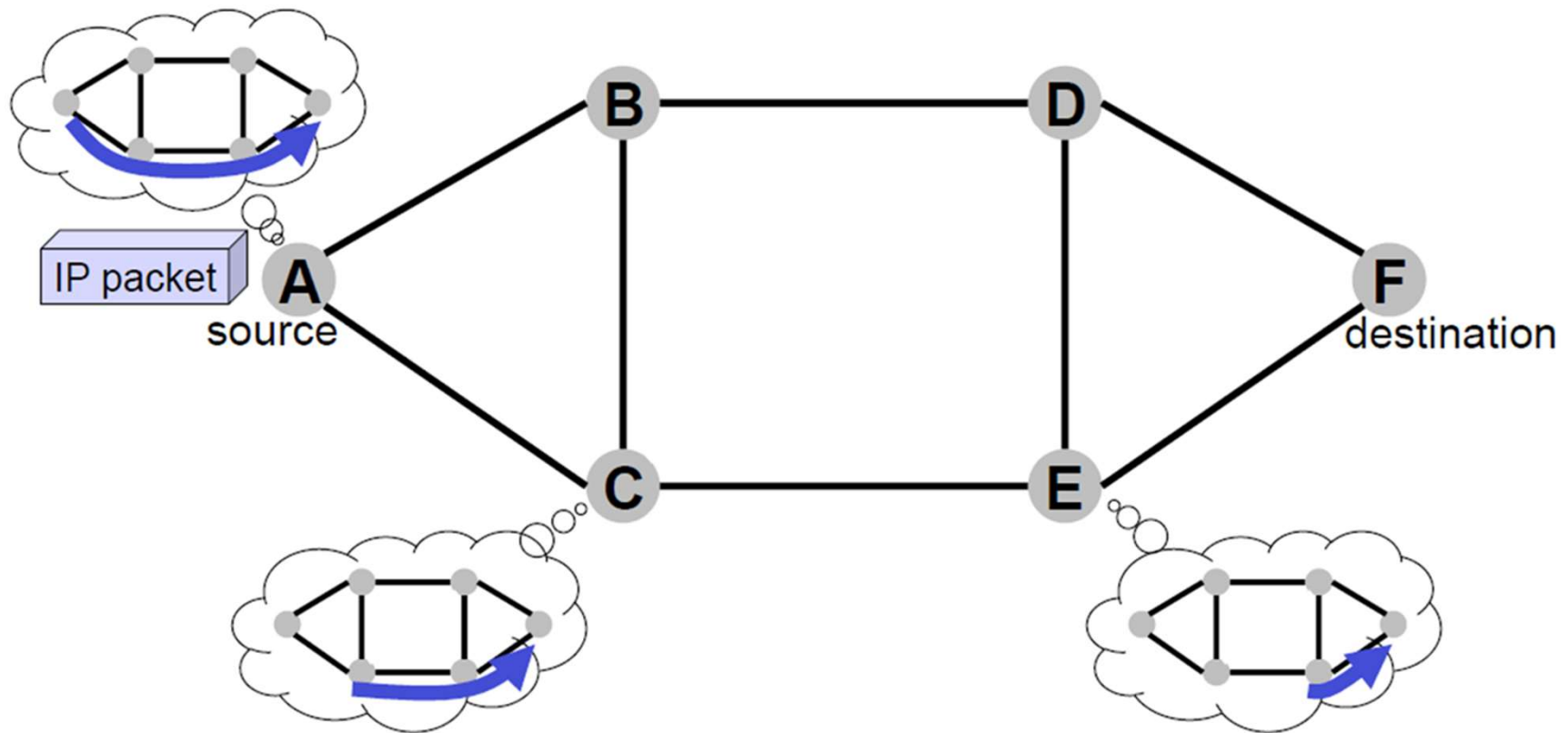
Link-state routing

- Each node uses Dijkstra algorithm to find the shortest paths to destinations



Link-state routing

- packet forwarding



Link-state routing

- Link-state packet forwarding
 - send LSP to all nodes in the network
 - compare: distance vector sends only to direct neighbors
 - Network-wide flooding

Flooding

- A sends LSP to all its neighbors
- Suppose node B receives A's LSP
- B checks the sequence number of A's LSP
 - If it is the newest,
 - B updates its routing table, resends the LSP to all its neighbors excluding node A
 - If it is not the newest
 - Discard the LSP

OSPF

- Open Shortest Path First
- A routing protocol based on link state routing
- Each router sends Link State Advertisement (LSA) packets
 - Periodically
 - On updates

OSPF

- LSA packet format

LS Age		Options		Type=1
Link-state ID				
Advertising router				
LS sequence number				
LS checksum		Length		
0	Flags	0	Number of links	
Link ID				
Link data				
Link type	Num_TOS	Metric		
Optional TOS information				
More links				

ID of the router Sending LSA

ID of the router at the end of this link

Disambiguate parallel links

Cost of the link

Summary

- Link state (OSPF)
 - Info exchanged: (neighbor ID, distance to neighbor)
 - Sent to: all nodes in the network (flooding)
 - Algorithm: Dijkstra
 - Pros: fast convergence, no loops
 - Cons: higher message and CPU overhead
- Distance vector (RIP)
 - Info exchanged: (destination, distance to destination)
 - Sent to: direct neighbors only
 - Algorithm: Bellman-Ford (distributed)
 - Pros: simple, low overhead
 - Cons: slow convergence, count-to-infinity problem